PROJECT

Submitted by- SARIF AHMAD

RTL2GDSII FLOW WITH SYNOPSYS TOOLS

a) Design 8x1 MUX for 8-inputs using the Verilog HDL

b) simulate it using VCS and Verdi

c) Generate GDS-II layout of the design using the Synopsys-flow scripts.

1.0 Report design Area

2.0 use clock-period 1-1.2ns and write clock constraints, where slack value is positive but less than 1.

A multiplexer (Mux) is a combinational logic circuits that selects one of several input data lines and forwards the selected input data to a single output line. It has multiple input lines carrying data and a single output line based on control or select inputs. A mux is often called a data selector. Typically, a multiplexer has 2^n input lines and n selection lines whose bit combinations determine which input is selected. The mux is a special and one of the most widely used standard circuit in complex digital design. A mux can also be known as Data selector or many to one circuit or universal logic converter or parallel to serial converter. Multiplexer play a crucial role in various digital systems, including communication system, data acquisition systems, complex digital logic circuits and microcontrollers and more.

Here, we are designing the **8x1 mux for 8-inputs** digital circuit. 8-input lines, **3 data select lines** and corresponding **one output line (8-bits).**

Design Table:

sel (selection lines)	sel (selection lines)	sel (selection lines)	out_mux (output)
0	0	0	in0 (8-bits)
0	0	1	in1 (8-bits)
0	1	0	in2 (8-bits)
0	1	1	in3 (8-bits)
1	0	0	in4 (8-bits)
1	0	1	in5 (8-bits)

1	1	0	in6 (8-bits)
1	1	1	in7 (8-bits)

Design code

```
// Design an 8x1mux for 8-bit inputs by using the Verilog code
module mux_8x1_8bit(
        input logic clk,
        input logic rst,
        input logic [7:0] in0, in1, in2, in3, in4, in5, in6, in7,
        input logic [2:0] sel,
        output logic [7:0] out_mux
);

logic [7:0] in0_reg, in1_reg, in2_reg, in3_reg, in4_reg, in5_reg, in6_reg, in7_reg;
logic [7:0] out_reg_mux;
logic [2:0] sel_reg;

always_ff @(posedge clk or posedge rst) begin
      if (rst) begin
              in0_reg <= 8'd0;
              in1_reg <= 8'd0;
              in2_reg <= 8'd0;
              in3_reg <= 8'd0;
              in4_reg <= 8'd0;
              in5_reg <= 8'd0;
              in6_reg <= 8'd0;
              in7_reg <= 8'd0;
              sel_reg <= 3'd0;
```

```
        end else begin
                in0_reg <= in0;
                in1_reg <= in1;
                in2_reg <= in2;
                in3_reg <= in3;
                in4_reg <= in4;
                in5_reg <= in5;
                in6_reg <= in6;
                in7_reg <= in7;
                sel_reg <= sel;
        end
end

always_comb begin
        unique case (sel_reg)
          3'd0: out_reg_mux = in0_reg;
          3'd1: out_reg_mux = in1_reg;
          3'd2: out_reg_mux = in2_reg;
          3'd3: out_reg_mux = in3_reg;
          3'd4: out_reg_mux = in4_reg;
          3'd5: out_reg_mux = in5_reg;
          3'd6: out_reg_mux = in6_reg;
          3'd7: out_reg_mux = in7_reg;
           default:out_reg_mux = 8'hFF;
endcase
end
always_ff @(posedge clk or posedge rst)begin
```

```
    if (rst)
        out_mux <= 8'd0;
    else
        out_mux <= out_reg_mux;
end
endmodule
```

Testbench

```
// testbench of 8x1mux for 8-bit inputs
module mux8x1_tb();
logic clk,rst;
logic [7:0] in0,in1,in2,in3,in4,in5,in6,in7;
logic [2:0] sel;
logic [7:0] out_mux;

mux_8x1_8bit
dut(.clk(clk),.rst(rst),.in0(in0),.in1(in1),.in2(in2),.in3(in3),.in4(in4),.in5(in5),.in6(in6),.in
7(in7),.sel(sel),.out_mux(out_mux));

initial
     clk=0;
always #5 clk=~clk;
initial begin
    in0 = 8'h00; in1 = 8'h11; in2 = 8'h22; in3 = 8'h33;
    in4 = 8'h44; in5 = 8'h55; in6 = 8'h66; in7 = 8'h77;
    sel = 3'b000;
    rst = 1;

    #10 rst = 0;
```

```
    for (int i = 0; i < 8; i++) begin
            sel = i;
            #10;
            $display("Time=%0t:rst=%b sel=%b,out_mux=%h",$time,rst,sel,out_mux);
    end
    #10 $display("last output: out_mux=%h", out_mux);
    #20 $finish;
end
initial begin
    $fsdbDumpfile("myfile.fsdb");
    $fsdbDumpvars(0,mux8x1_tb);
end
endmodule
```

Run command

```
vcs -sverilog mux_8x1_8bit.sv mux_8x1_8bit_tb.sv -kdb -lca -debug_access+all -full64
```

Simulation Result

Output

Input changes sampled at first posedge clock edge and output will be available at 2nd posedge clock edge means one-clock delay

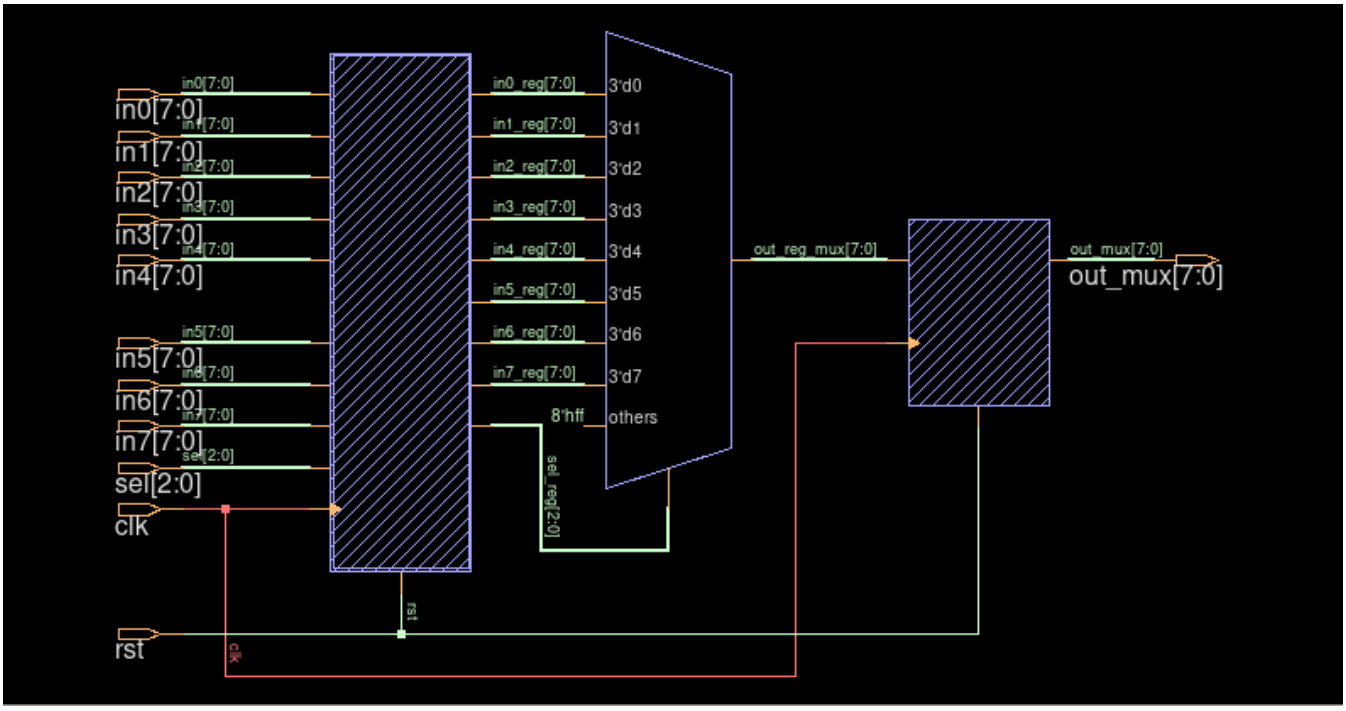
```
Time=20:rst=0 sel=000,out_mux=00
Time=30:rst=0 sel=001,out_mux=00
Time=40:rst=0 sel=010,out_mux=11
Time=50:rst=0 sel=011,out_mux=22
Time=60:rst=0 sel=100,out_mux=33
Time=70:rst=0 sel=101,out_mux=44
Time=80:rst=0 sel=110,out_mux=55
```

Time=90:rst=0 sel=111,out_mux=66

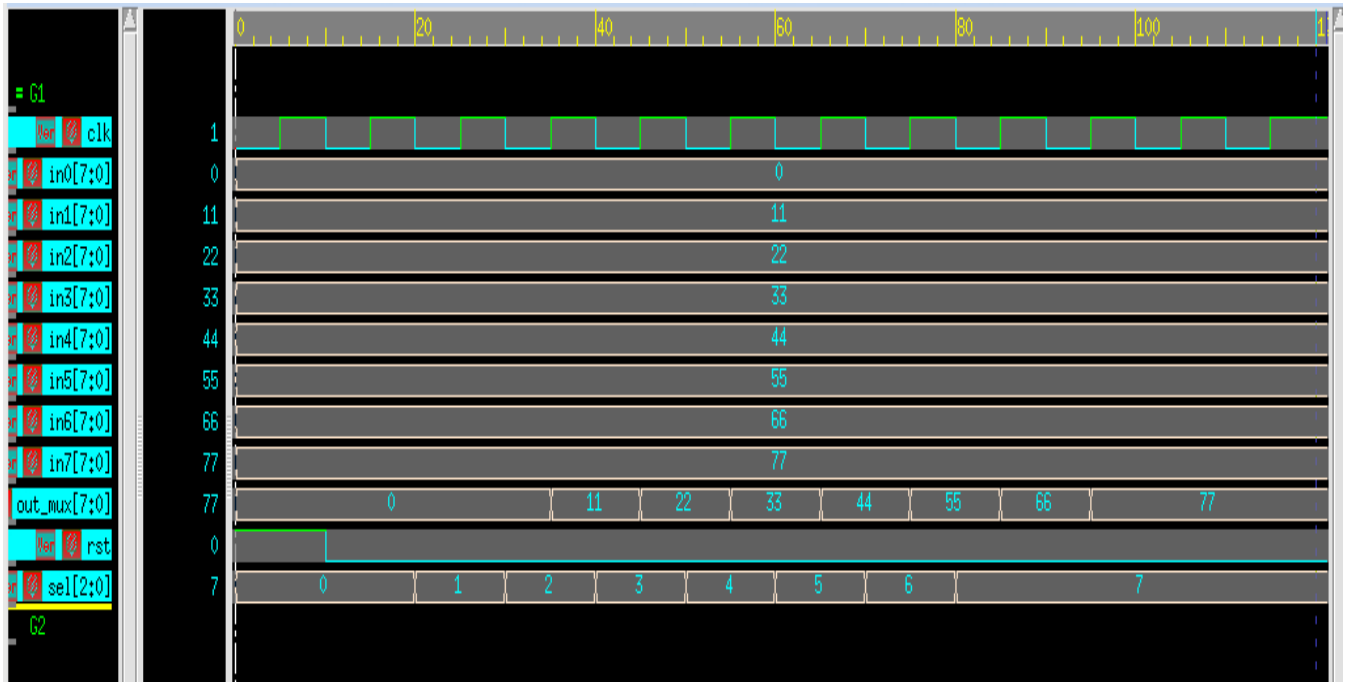
last output: out_mux=77

$finish at simulation time 120

Schematic Design



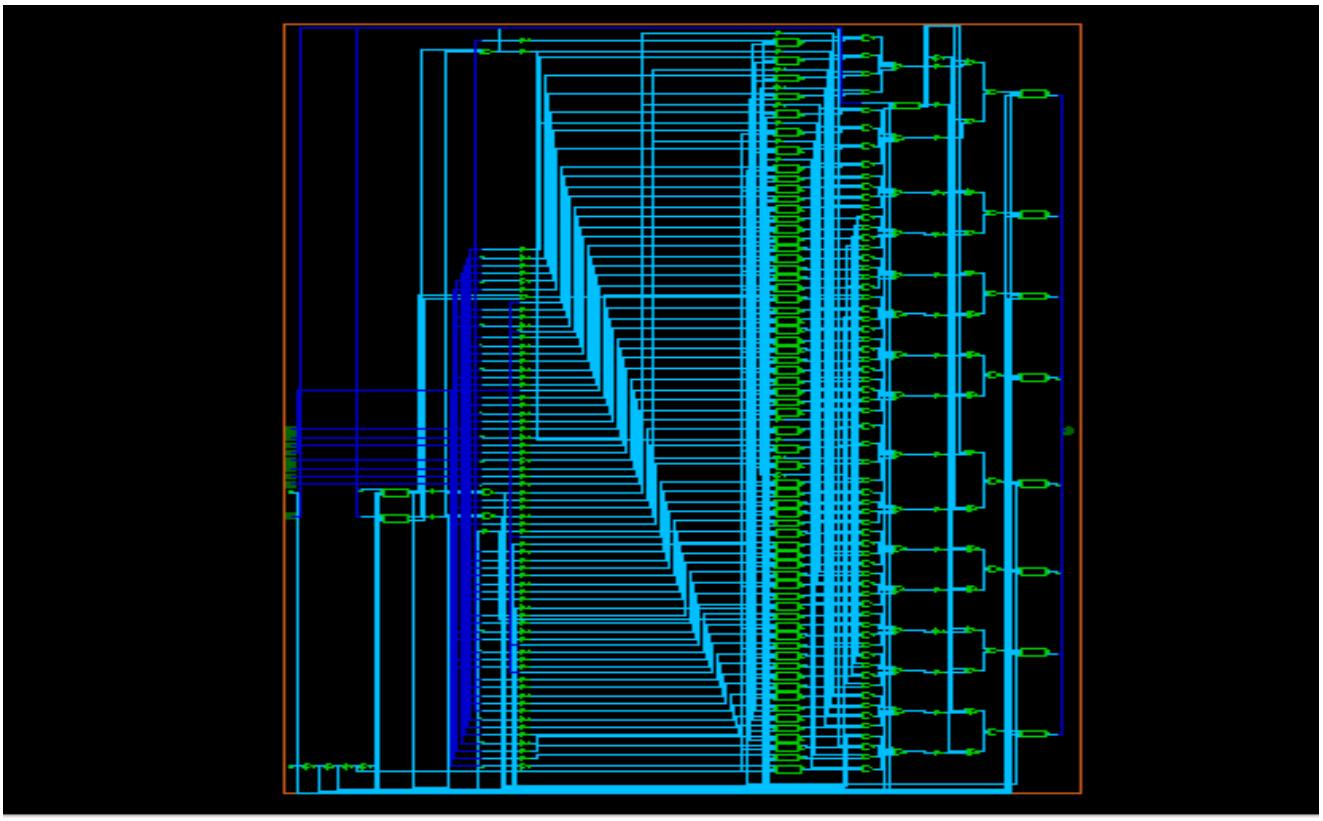
waveform



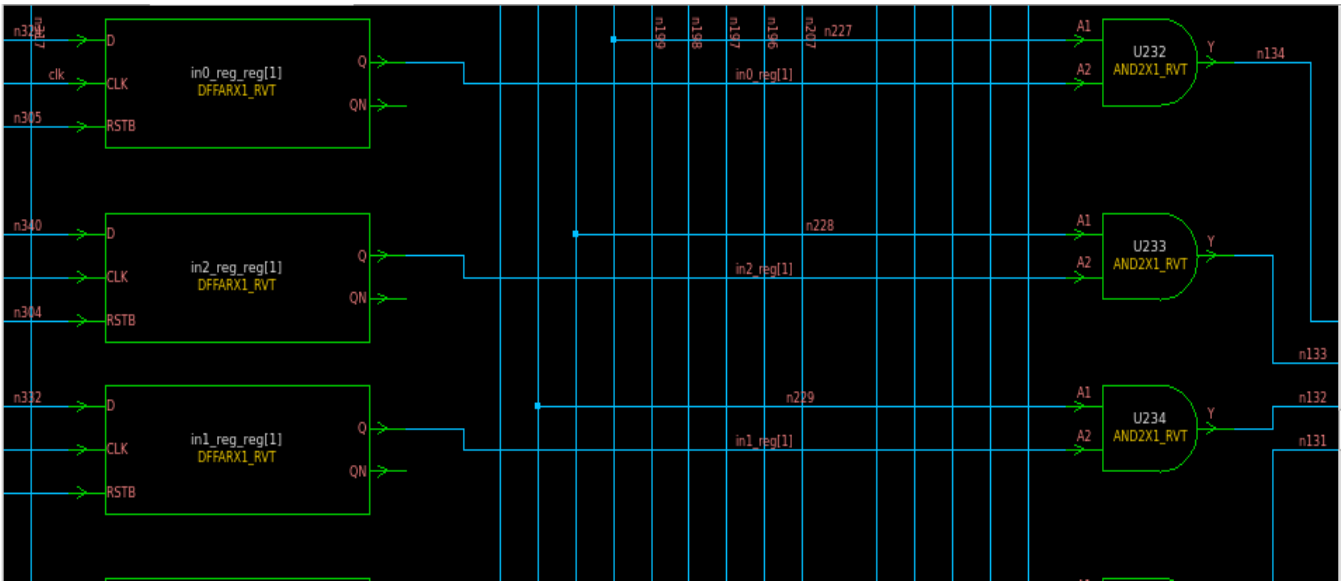
c) Generate GDS-II layout of the design using the Synopsys-flow scripts.



(Schematic View of 8x1mux for 8-inputs- by Synopsys Design Compiler (SDC))



(This is the internal schematic view of synthesizable circuits)

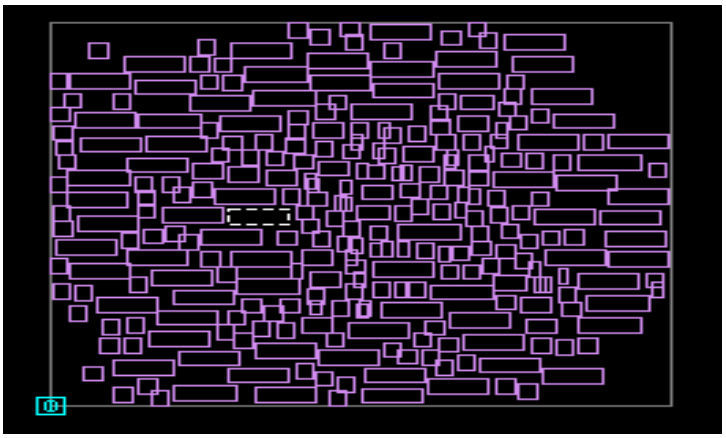


(By using the Synopsys Design Compiler (SDC))

Floorplan of 8x1 MUX for 8-inputs–

Floorplanning in VLSI is a crucial step and most important process in the physical design of an integrated circuit. Floorplanning is the process of placing blocks/macros in the chip/core area. In this step we have netlists which describes the design and the various blocks of the design and the interconnection between the blocks. The netlist is the logical description of the ASIC design. Floorplan is the physical description of ASIC design. In floor planning, we are mapping the logical description of the design to the physical description.

In this step, we define the size of the chip/block, allocates power routing resources, place the hard macros, and reserve space for standard cells. A good floorplan can be means overall improve the performance of chip making placement, CTS, routing, and timing closure easy and a bad floorplan can directly impact the area, power consumption, performance and reliability of the chip.

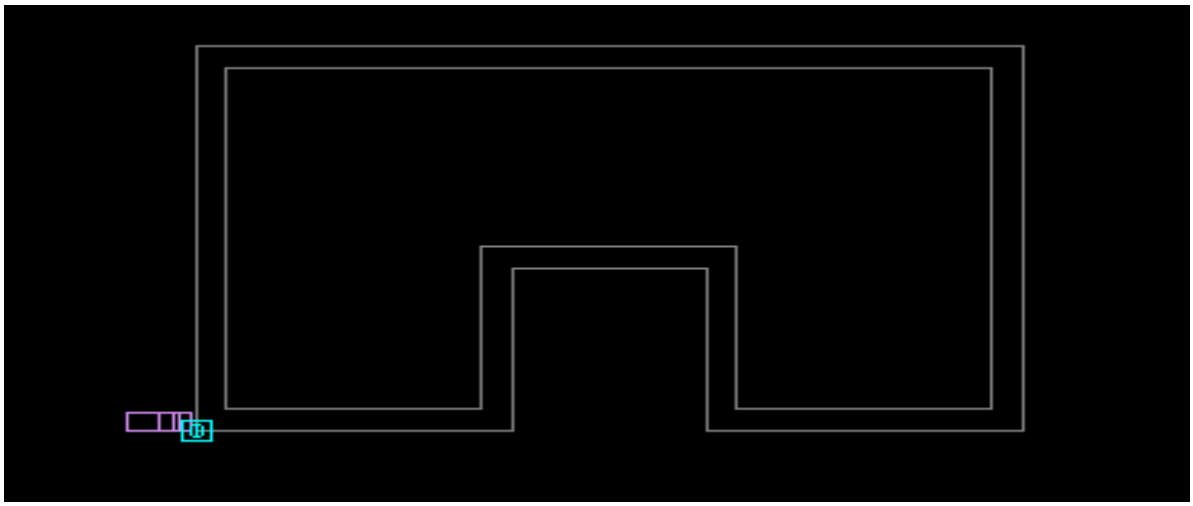


icc2_shell> initialize_floorplan -core_utilization 0.5

Removing existing floorplan objects

Creating core...

Core utilization ratio = 50.35%



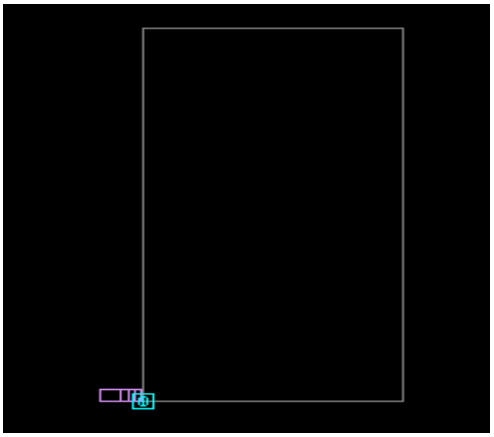
icc2_shell> initialize_floorplan -shape U -core_utilization 0.7 -orientation S -core_offset 2 -coincident_boundary true

Removing existing floorplan objects

Creating core...

Info: For non-rectangle shape, support to compare core_area's length with macros' max width and height.

Core utilization ratio = **72.47%**

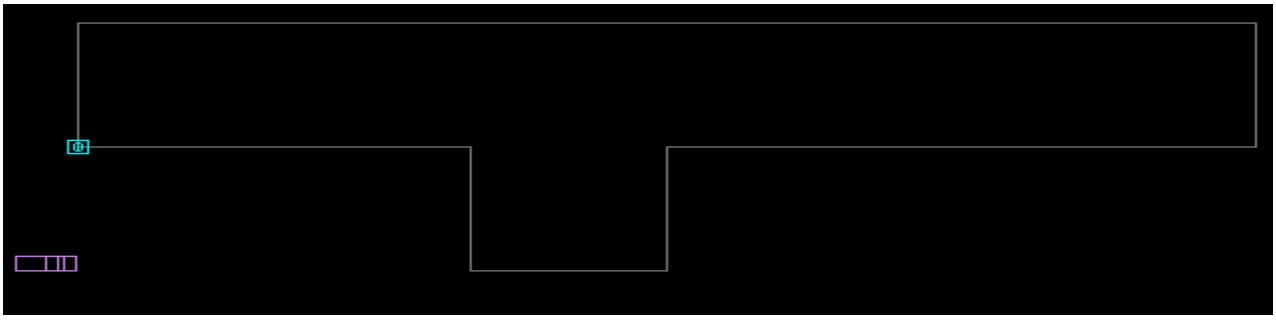


icc2_shell> initialize_floorplan -side_ratio {50 100} -shape R

Removing existing floorplan objects

Creating core...

Core utilization ratio = 70.87%



icc2_shell> initialize_floorplan -shape T -side_ratio {1 3 1 1 1 2}

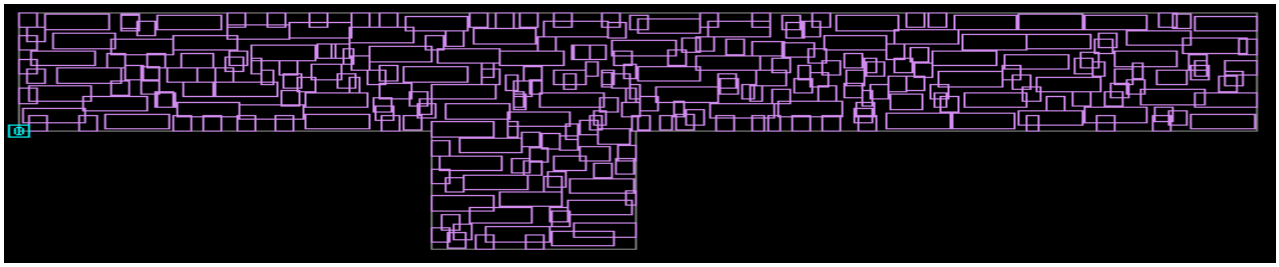
Removing existing floorplan objects

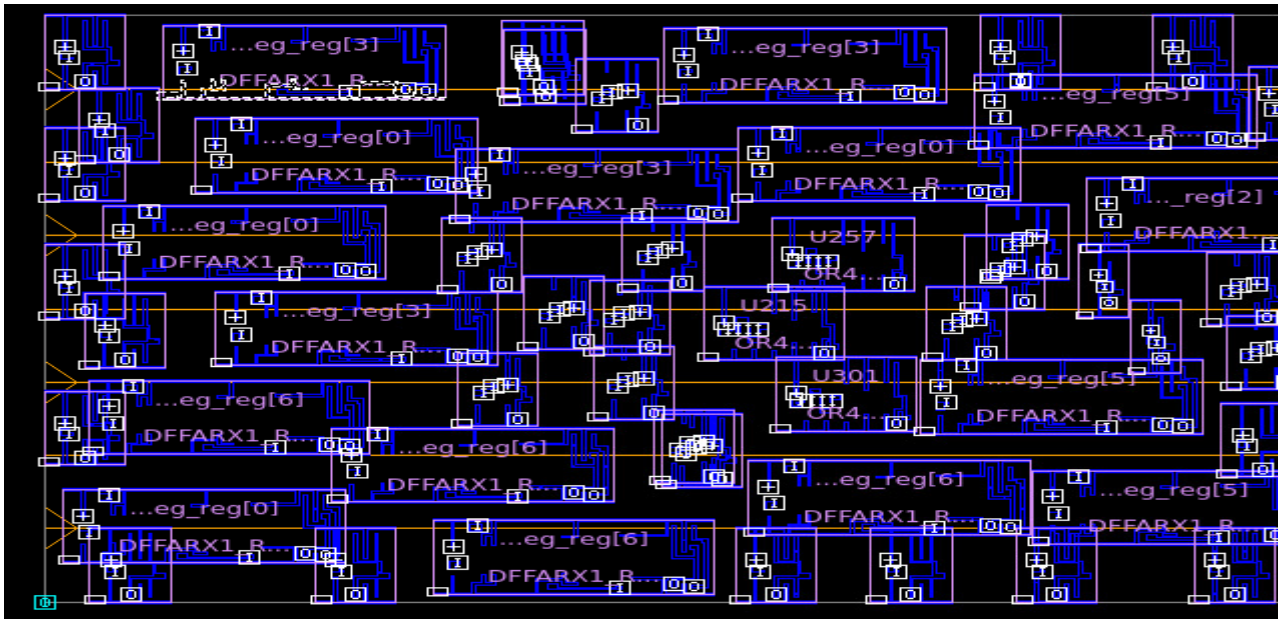
Creating core...

Info: For non-rectangle shape, support to compare core_area's length with macros' max width and height.

Core utilization ratio = 73.38%

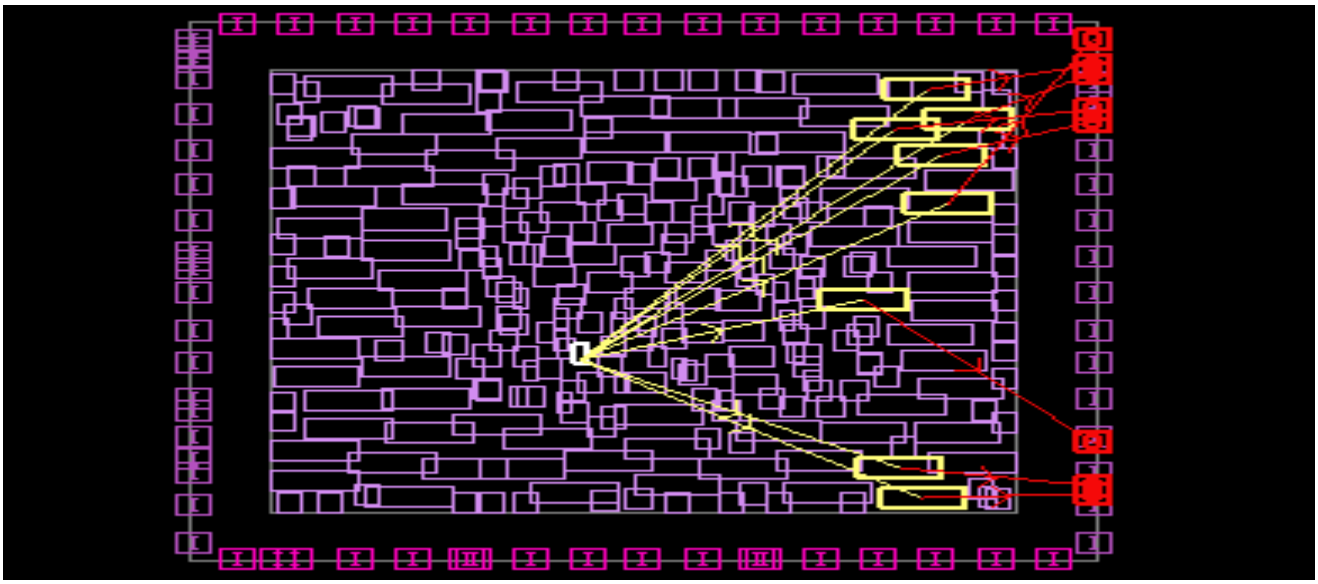
icc2_shell> create_placement -floorplan





(Frame view- information regarding the pins like I/O)

Flylines



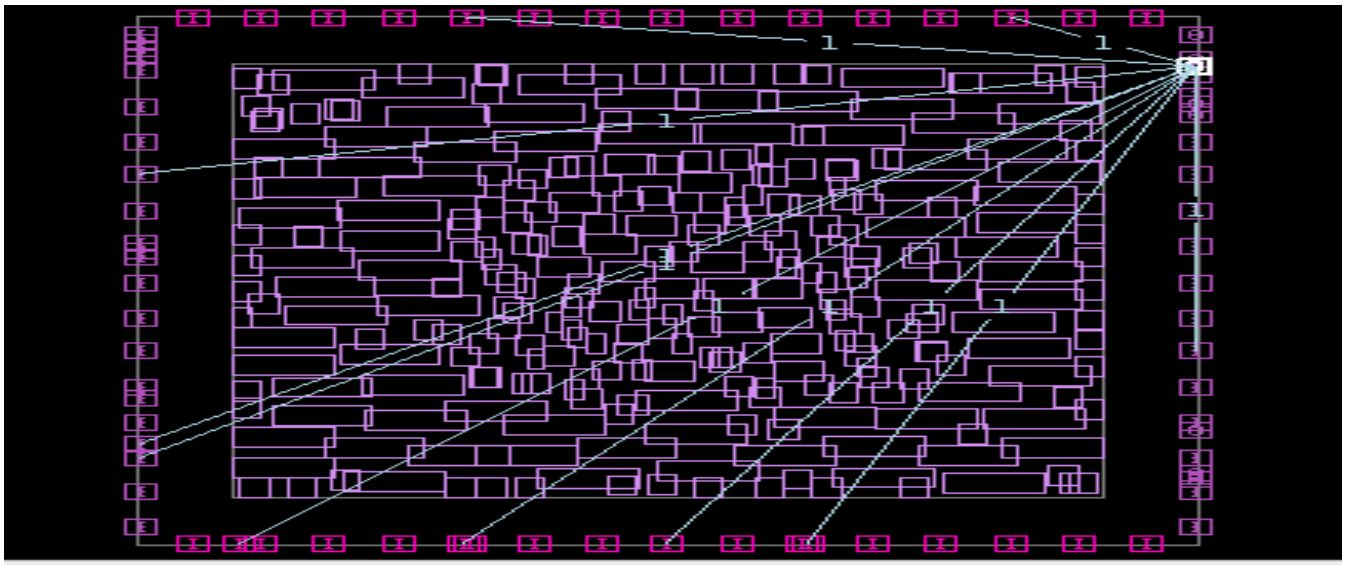
Register Tracing (How data is transferring from one register to another register inside the chip)

Register Tracing Flyline

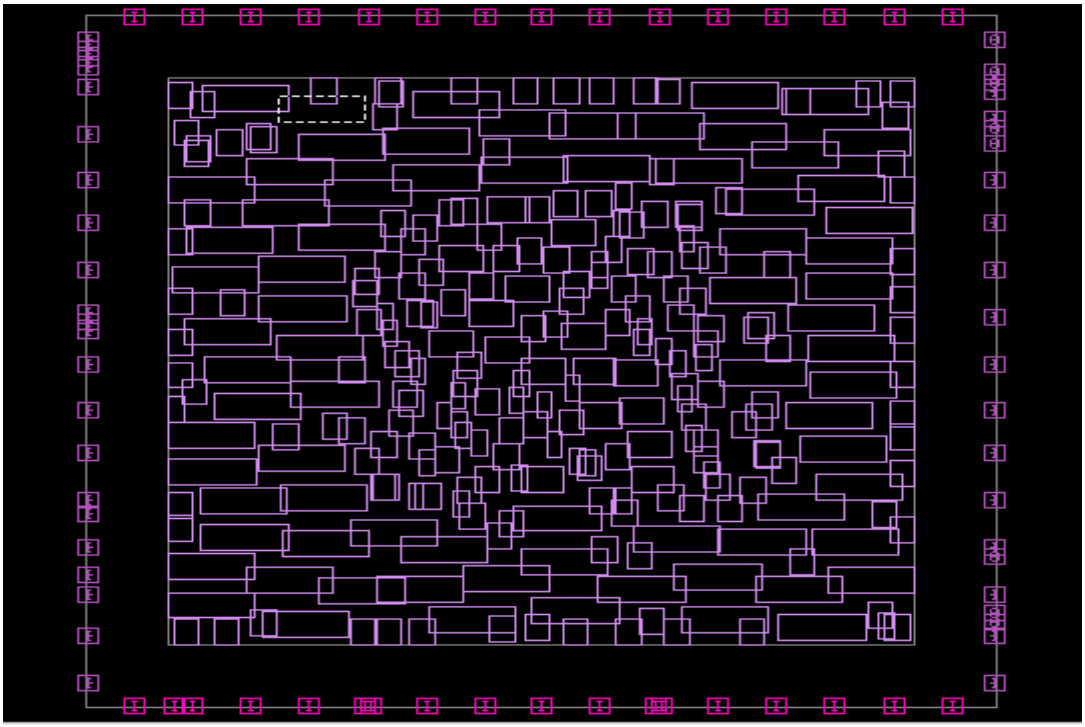
From: in2_reg_reg[5]

To: out_mux_reg[5]

Level: 2



(Data Flow Flylines: data is travelling b/w more than two macros/pin)



(Floorplan of 8x1mux)

Command used in tool-

set PDK_PATH ./../ref
./../ref/lib/ndm/saed32rvt_c.ndm
../DC/results/mux_8x1_8bit.mapped.v

read_verilog -library MUX -design mux -top mux ../DC/results/mux_8x1_8bit.mapped.v

./../ref/lib/ndm/saed32rvt_c.ndm

icc2_shell> history

1 set PDK_PATH ./../ref

2 start_gui

3 create_lib -ref_libs ./../ref/lib/ndm/saed32rvt_c.ndm MUX

8 open_lib MUX/

9 start_gui

10 read_verilog -library MUX -design mux -top mux
../DC/results/mux_8x1_8bit.mapped.v

11 link_block

16 open_lib /ve/po_home/b23n_sarif_727/internship_may_jun_25/ICCII/MUX

17 open_block /ve/po_home/b23n_sarif_727/internship_may_jun_25/ICCII/MUX: MUX_FP.design

18 link_block

19 initialize_floorplan

20 create_placement -floorplan

21 report_design

23 initialize_floorplan -core_utilization 0.5

24 create_placement -floorplan

25 report_design

66 initialize_floorplan -shape U -core_utilization 0.7 -orientation S -core_offset 2 -coincident_boundary true

70 initialize_floorplan -side_ratio {50 100} -shape R

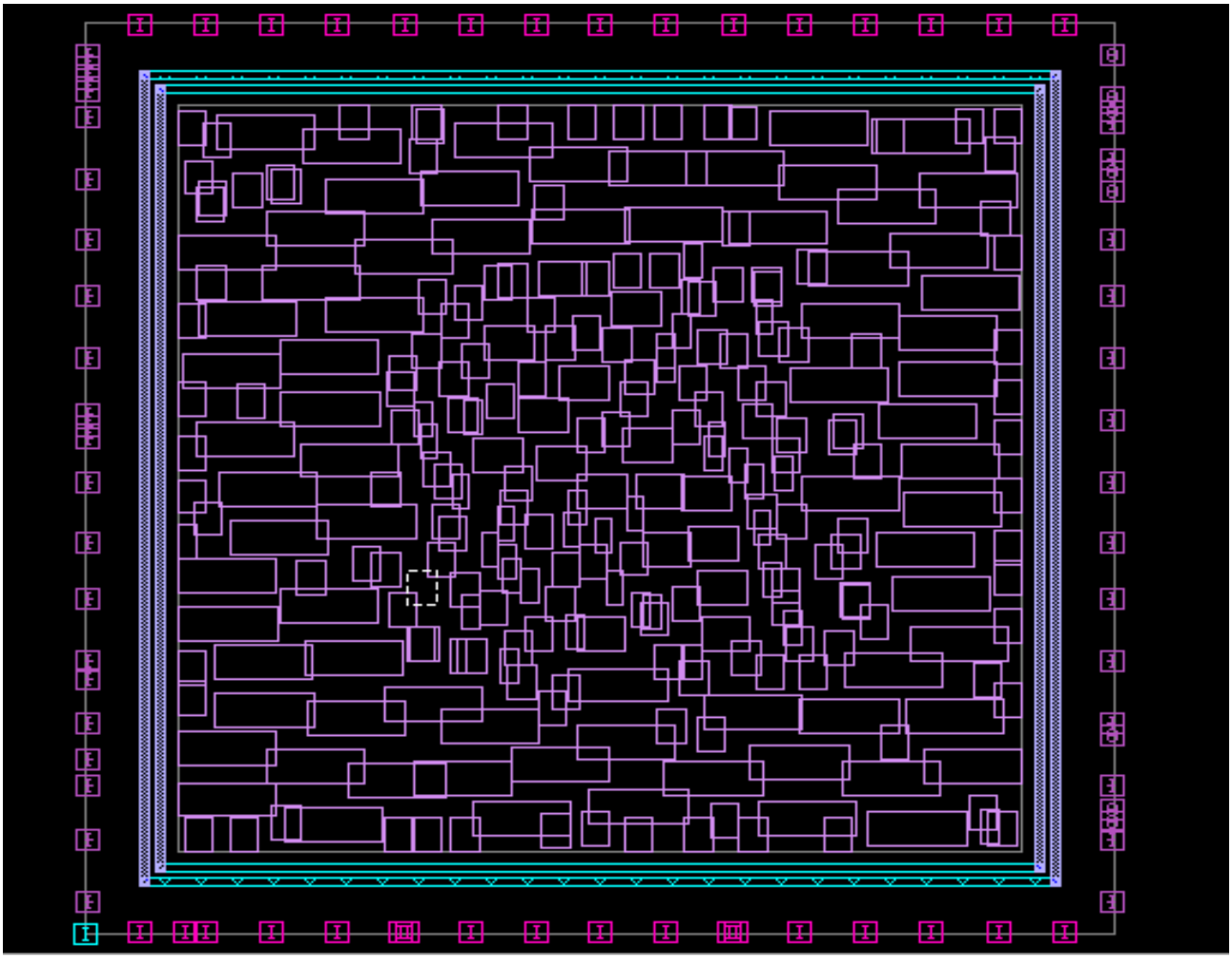
Power Planning

Power planning in VLSI is a crucial process that involves designing the power deliver network (PDN) and provide adequate and stable power to every macro, standard cells, and all other cells are present in the design. Power and ground nets are usually laid out on the metal layers. In this create power and ground structure for both I/O pads and core logic. The I/O pads power and ground buses are built into the pad itself and will be connected by abutment. It focuses on distributing power (VDD) and ground (VSS) efficiently, managing voltage drops, and minimizing power consumption.

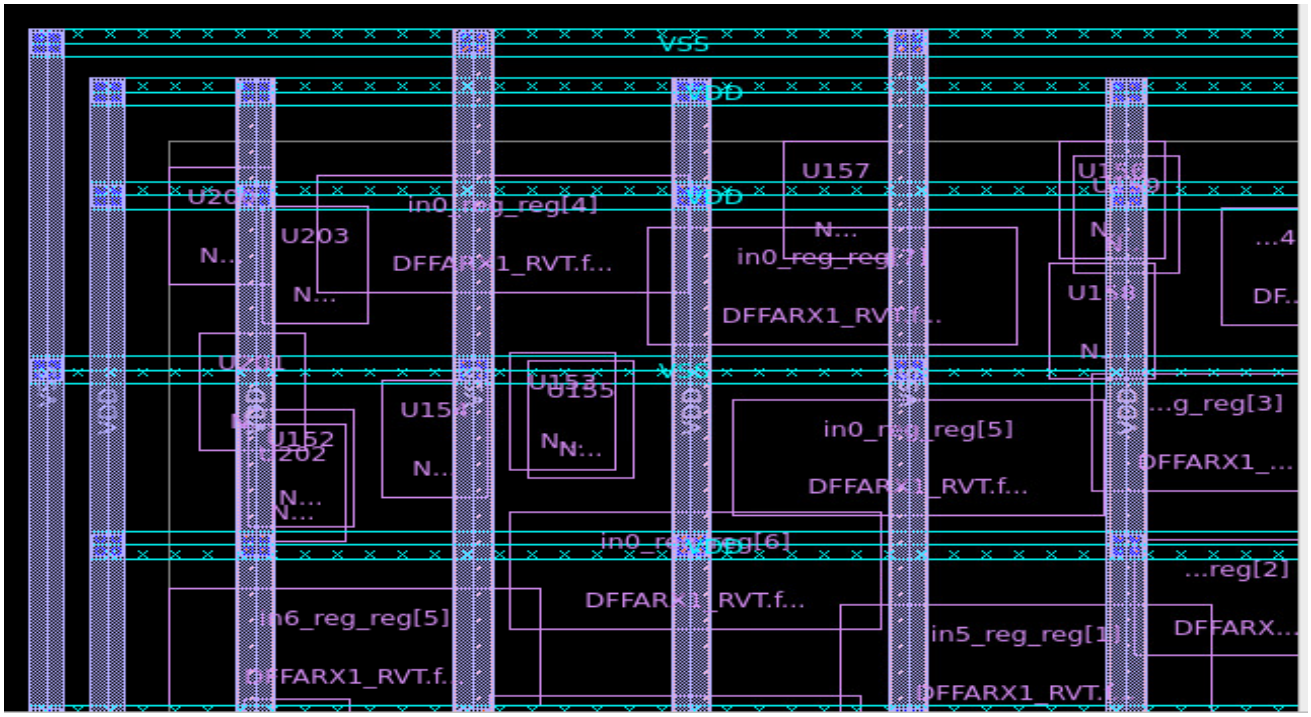
Cell power and ground is internal to core logic. These are called power and ground strips that repeat at regular intervals across the logic or specified region, within the design. Each of these strips run both vertically and horizontally at regular interval then this is called power mesh. Power ring is designed around the core. As power consumption (static and dynamic) increases the distance of power and ground straps interval increase to reduce overall voltage drop, thereby improving performance. Objective of power planning is to meet IR drop budget. Power planning involves-calculating number of power pins required number of rings and straps, width of rings and straps and IR drop.

There are three level of power distributions-

1.0 power rings: carries VDD and VSS around the chip.
2.0 Power straps: carries VDD and VSS from the rings across the chip.
3.0 Power rails: connect VDD and VSS to the standard cells.



(This is a ring-pattern of Power planning)



(Internal view of mesh-pattern of mux and power-rail)

Command used in tool-

icc2_shell> history

1 open_lib MUX

2 open_block Mux_FP_1

3 start_gui

4 link_block

remove__routes -net_types {power ground} -ring -stripe

compile_pg -undo

icc2_shell> history

1 open_lib MUX/

2 open_block MUX_FP_1

3 start_gui

4 link_block

5 create_net -power {VDD}

6 create_net -ground {VSS}

7 connect_pg_net -all -automatic

```
8  connect_pg_net -all -automatic

9  create_pg_ring_pattern core_ring -horizontal_layer M7 -horizontal_width 4 -horizontal_spacing 2 -vertical_layer M8 -vertical_width 4 -vertical_spacing 2

10  set_pg_strategy core_pg_ring -core -pattern {{name: core_ring} {nets: {VDD VSS}} {offset: {1 1}}}

11  compile_pg -strategies core_pg_ring

12  remove_pg_patterns core_ring

13  remove_pg_patterns -all

14  remove_pg_strategies -all

15  compile_pg -undo

16  create_pg_ring_pattern core_ring -horizontal_layer M7 -horizontal_width .4 -horizontal_spacing .3 -vertical_layer M8 -vertical_width .4 -vertical_spacing .3

17  set_pg_strategy core_pg_ring -core -pattern {{name: core_ring} {nets: {VDD VSS}} {offset: {.5 .5}}}

18  compile_pg -strategies core_pg_ring

19  create_pg_mesh_pattern core_mesh -layers {{{vertical_layer: M6} {width: .4} {spacing: interleaving} {pitch : 5} {offset: 1}} {{horizontal_layer: M7} {width: .4} {spacing: interleaving} {pitch: 5} {offset: 1} }}

20  create_pg_mesh_pattern core_mesh -layers {{{vertical_layer: M6} {width: .4} {spacing: interleaving} {pitch : 5} {offset: 1}} {{horizontal_layer: M7} {width: .4} {spacing: interleaving} {pitch: 5} {offset: 1} } {{vertical_layer: M8} {width: .45} {spacing: interleaving} {pitch: 5} {offset: 1}}}

21  set_pg_strategy core_power_mesh -pattern {{pattern: core_mesh} {nets: VDD VSS}} -core -extension {stop:innermost_ring}

22  compile_pg -strategies core_power_mesh

23  create_pg_std_cell_conn_pattern std_cell_rail -layer {M1} -rail_width 0.06

24  set_pg_strategy rail_strat -core -pattern {{name: std_cell_rail}} {nets: VDD VSS}}

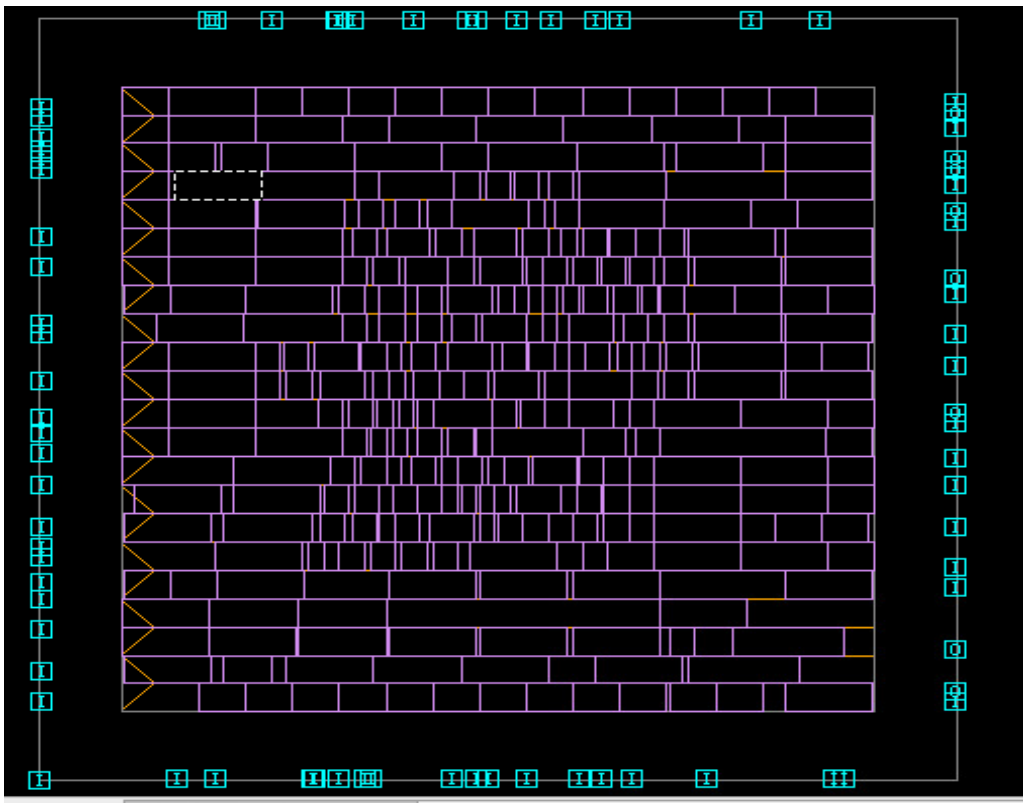
25  set_pg_strategy rail_strat -core -pattern {{name: std_cell_rail} {nets: VDD VSS}}

26  compile_pg -strategies rail_strat
```

Placement

Placement is a very important stage of physical design and it is the process of placing the standard inside the core boundary in in an optimal location. Overall report_qor and report_timing of the design greatly depends on the fact that how well placement is done. The tool tries to place the standard cell in such a way that the design should have minimum congestions and minimum timing DRC's.

Every PnR tool provides various commands/switches so that users can optimize the design in a better way in terms of timing, congestion, area, and power as per their requirements. Based on the preferences set by the user, the tool tray to place and optimize it for better QOR. Placement does not place only the standard cells present in the synthesized netlist but also places many physical only cells and adds buffers/inverters as per the requirement to meet the timings, DRV, and foundry requirements. Here are the basic steps which the tool performs during the placement and optimization stage.



(Placement of cell in 8x1 mux)

Command used in tool-

```
set PDK_PATH ./../ref
2 set mode1 "func"
3 set corner1 "nom"
4 set scenario1 "${mode1}::${corner1}"
```

5 remove_modes -all
6 open_lib MUX/
7 open_block MUX_PP
8 remove_modes -all
9 remove_corners -all
10 remove_scenarios -all
create_ mode $mode1
13 create_mode $mode1
14 create_corner $corner1
15 create_scenario -name func::nom -mode func -corner nom
current_mode func
18 current_corner nom
19 current_scenario func::nom
20 source ./../mux_8bit.sdc
21 set_dont_use [get_lib_cells */FADD*]
22 set_dont_use [get_lib_cells */HADD*]
23 set_dont_use [get_lib_cells */AO*]
24 set_dont_use [get_lib_cells */OA*]
25 set_dont_use [get_lib_cells */NAND*]
26 set_dont_use [get_lib_cells */XOR*]
27 set_dont_use [get_lib_cells */XNOR*]
28 set_dont_use [get_lib_cells */NOR*]
29 set_dont_use [get_lib_cells */MUX*]
30 current_scenario
31 set parasitic1 "p1"
set tluplus_file$parasitic1 "$PDK_PATH/tech/star_rcxt/saed32nm_1p9m_Cmax.tluplus"
33 set layer_map_file$parasitic1
"$PDK_PATH/tech/star_rcxt/saed32nm_tf_itf_tluplus.map"
34 set parasitic2 "p2"
35 set tluplus_file$parasitic2
"$PDK_PATH/tech/star_rcxt/saed32nm_1p9m_Cmin.tluplus"
36 set layer_map_file$parasitic2
"$PDK_PATH/tech/star_rcxt/saed32nm_tf_itf_tluplus.map"
read_parasitic_tech -tlup $tluplus_filep1 -layermap $layer_map_filep1 -name p1

read_parasitic_tech -tlup $tluplus_filep2 -layermap $layer_map_filep2 -name p2
44 set_parasitic_parameters -late_spec $parasitic1 -early_spec $parasitic2
set_app_options -name place.coarse.continue_on_missing_scandef -value true

40 place_pins -self

42 place_opt

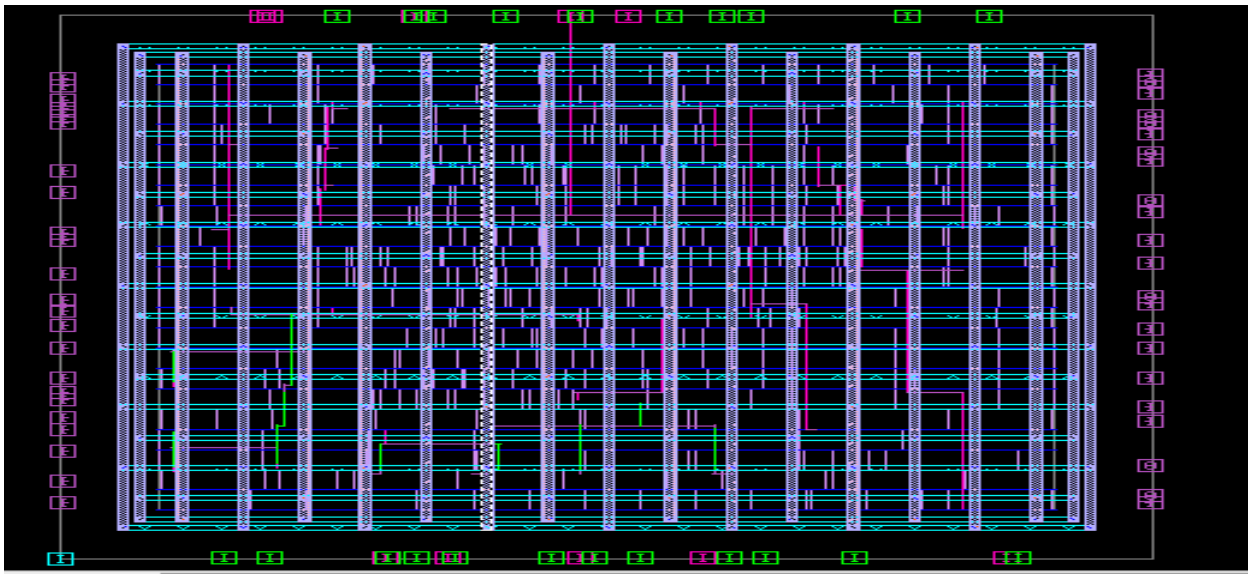
43 report_timing

```
44  report_timing -significant_digits 5
45  report_timing -significant_digits 5 -delay_type min
46  legalize_placement
```

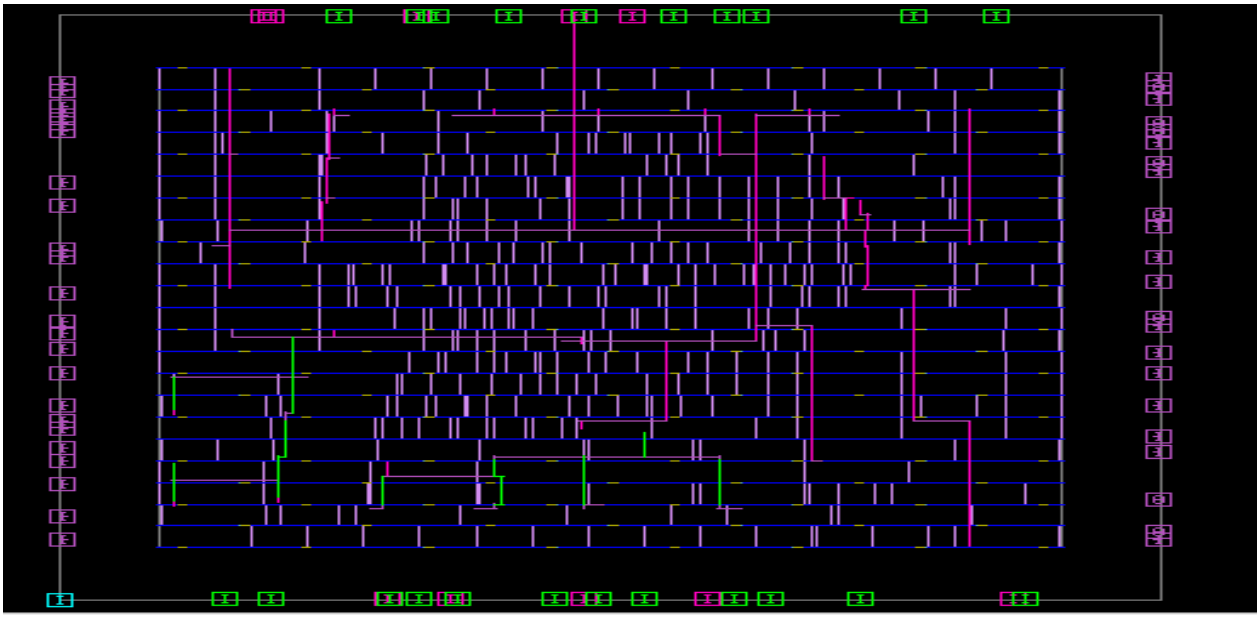
CTS (Clock tree synthesis)-

Clock Tree Synthesis (CTS) is one of the most important critical stages in VLSI backend place and Route (PnR). It involves distributing the clock signal from its source to all **clock sinks** (such as flip-flops and latches) using buffers and inverters, with minimal clock skew and insertion delay.

During placement stage, clocks are treated as **ideal (having zero delay)**. But in real silicon, clocks travel through metal and buffers like any other signal, and need a well-balanced **clock tree** to avoid timing violations. By minimizing clock skew, CTS ensures that all sequential elements receive the clock signal at approximately the same time, preventing timing violations. This synchronization is critical for maintaining the correct timing relationships across the design. The network often consumes a significant portion of a chip's power. Hence, CTS also plays a key role in optimizing the clock tree to reduce power consumption by intelligently inserting buffers and minimizing unnecessary toggling. Proper clock distribution is essential for the correct and reliable operation of any digital circuit. CTS effectively transforms a design from an ideal-timed model into a real-world clocked system, making it ready for subsequent **routing** and **signoff stages**.



(Signal layer global routing view of connection)



(Actual connection: Meta layer track detailed routing view)

Longest M1 wire length: 3.8

Longest M2 wire length: 16.2

Longest M3 wire length: 17.6

Longest M4 wire length: 3.6

Longest M5 wire length: 0.0

Longest M6 wire length: 0.0

Longest M7 wire length: 0.0

Longest M8 wire length: 0.0

Longest M9 wire length: 0.0

Longest MRDL wire length: 0.0

Finish DR since reached 0 DRC

Command used in tool-

icc2_shell> history

1 open_lib MUX/

2 open_block MUX_PLACE

3 start_gui

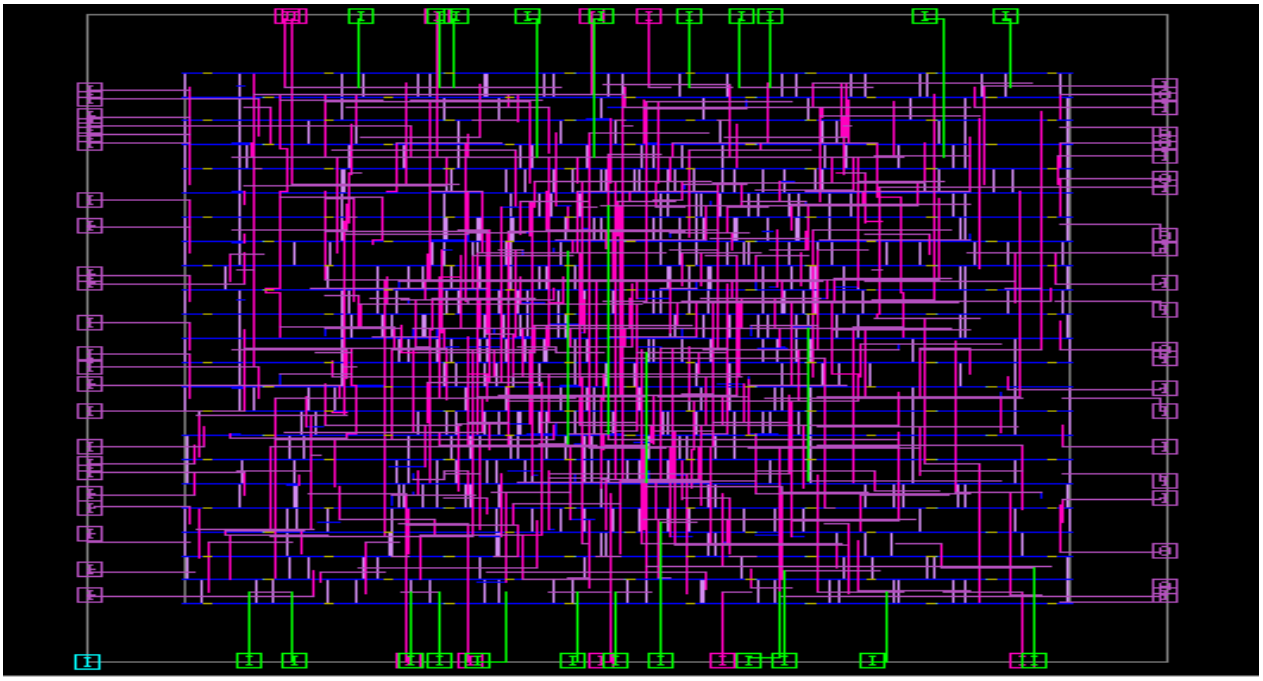
4 link_block

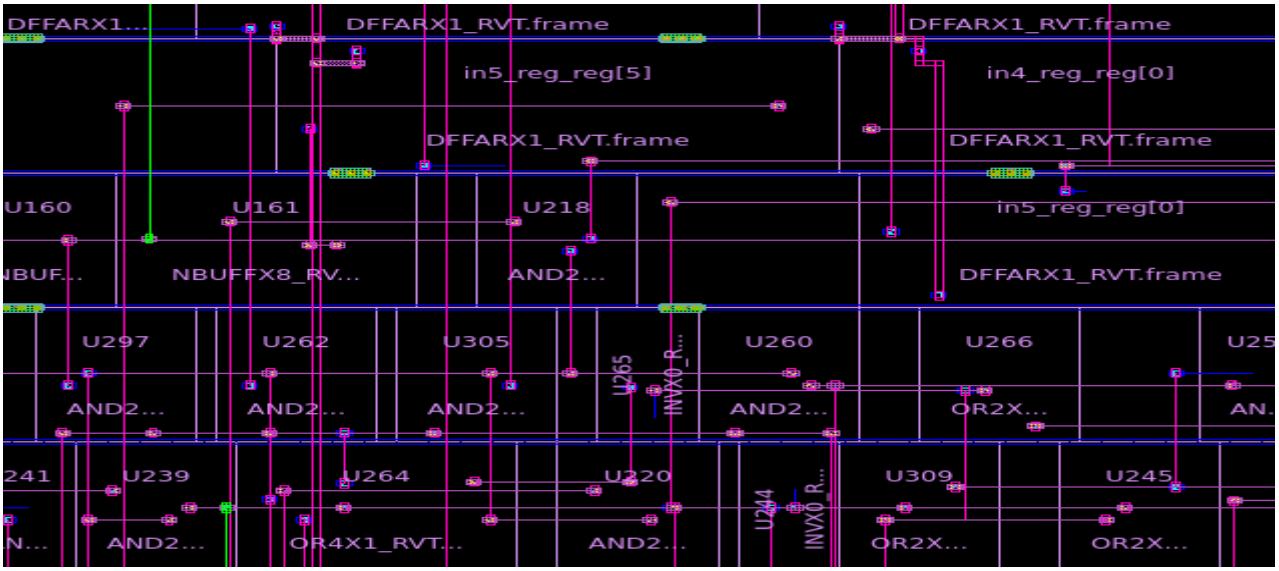
```
5  check_design -checks pre_clock_tree_stage
9  synthesize_clock_tree
10 set_app_options -name cts.optimize.enable_local_skew -value true
11 set_app_options -name cts.compile.enable_local_skew -value true
12 set_app_options -name cts.compile.enable_global_route -value true
13 set_app_options -name clock_opt.flow.enable_ccd -value true
15 clock_opt -to build_clock
16 report_timing
17 report_timing -delay_type min
18 all_outputs
19 report_timing -to out_mux[7]
22 report_timing
23 clock_opt -from route_clock -to route_clock
24 clock_opt
```

Routing

Routing is the crucial step of establishing physical connections between various components/blocks on a chip, like logic gates, transistors, and memory cells, using metal layers. It follows placement and aims to create electrical connections based on the logical connections defined in the netlist. The tool depends on this information to electrically complete all connections. The process involves determining the exact paths for these connections, optimizing wire length, minimizing critical path delay, and ensuring no design rule violations (DRC), minimize the number of via's, no LVS error, minimum routing area and minimizing the congestion hotspots.

In **routing stage**, metal and vias are used to create the electrical connection in layout, to complete all connections defined by the netlists. Routing is often divided into global and detailed routing.in global routing, which determines the general path for each net, and detailed routing, which defines the precise path and layer assignment. Routing also focuses on meeting timing requirements (setup and hold times) and ensuring signal integrity by minimizing RC delay and crosstalk, optimizing the power and performance.





(Global & detailed routing)

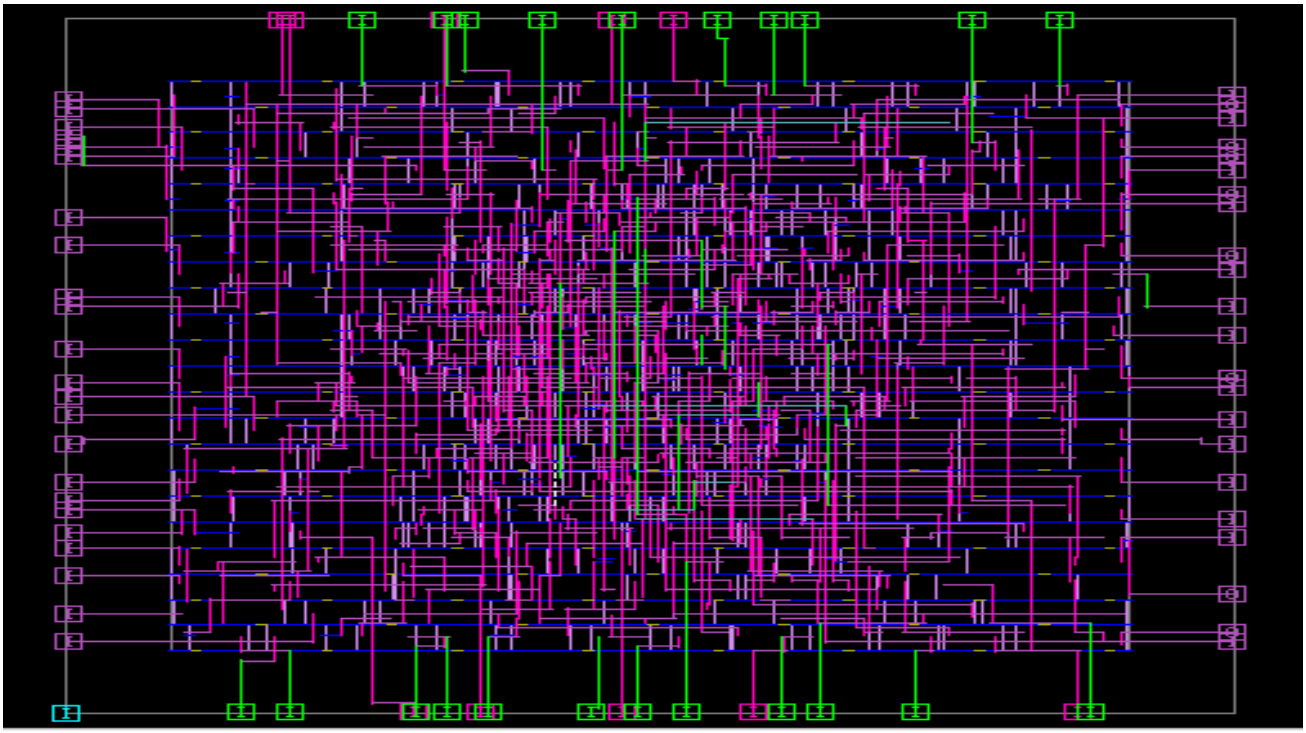
icc2_shell> route_track

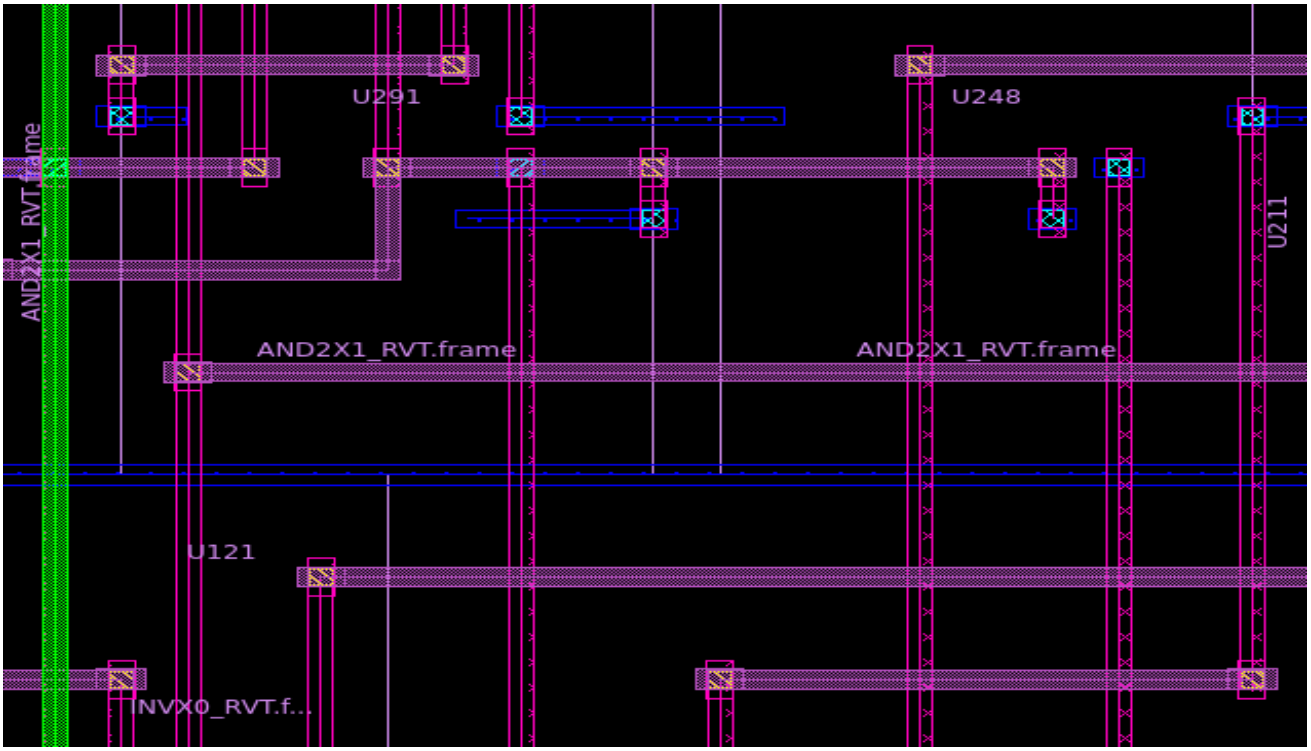
icc2_shell> route_detail

DRC-SUMMARY:

@@@@@@@ TOTAL VIOLATIONS = 0

Total number of DRCs = 0





(Assign with track/metals layers)

Command used in tool-

icc2_shell> history

1 open_lib MUX/

2 open_block MUX_CTS

```
  3  start_gui
  4  link_block
  5  clock_opt
  6  set_app_options -name route.global.timing_driven -value true
  7  set_app_options -name route.global.crosstalk_driven -value false
  8  set_app_options -name route.global.timing_driven -value true
  9  set_app_options -name route.global.crosstalk_driven -value false
 10  set_app_options -name route.track.timing_driven -value true
 11  set_app_options -name route.track.crosstalk_driven -value true
 12  set_app_options -name route.detail.timing_driven -value true
 13  set_app_options -name route.detail.save_after_iterations -value false
 14  set_app_options -name route.detail.force_max_number_iterations -value false
 15  set_app_options -name route.detail.antenna -value true
 16  set_app_options -name route.detail.antenna_fixing_preference -value use_diodes
 17  set_app_options -name route.detail.diode_libcell_names -value
*/ANTENNA_RVT
 18  route_global
 19  route_track
 20  route_detail
 21  route_opt
 23  report_timing
```

Simulation Results

Report design Area

Report_timing

icc2_shell> report_timing -to out_mux[3] -significant_digits 6

****

Report : timing

```
        -path_type full
        -delay_type max
        -max_paths 1
        -report_by design
Design : mux_8x1_8bit
Version: V-2023.12-SP4
Date   : Mon Jul  7 15:34:59 2025
****************************************

  Startpoint: out_mux_reg[3] (rising edge-triggered flip-flop clocked by clk)
  Endpoint: out_mux[3] (output port clocked by clk)
  Mode: func
  Corner: nom
  Scenario: func::nom
  Path Group: **reg2out_default**
  Path Type: max

  Point                                   Incr       Path
  -----------------------------------------------------------------------------
  clock clk (rise edge)                   0.000000   0.000000
  clock network delay (propagated)        0.071621   0.071621

  out_mux_reg[3]/CLK (DFFASX1_RVT)        0.000000   0.071621 r
  out_mux_reg[3]/QN (DFFASX1_RVT)         0.070114   0.141735 f
  out_mux[3] (out)                        0.000019   0.141754 f
  data arrival time                                  0.141754
```

clock clk (rise edge)	1.200000	1.200000
clock network delay (propagated)	0.000935	1.200935
clock uncertainty	-0.050000	1.150935
output external delay	-1.000000	0.150935
data required time		0.150951

---

data required time	0.150951
data arrival time	-0.141754

---

slack (MET)	**0.009197**

1

Finally, we obtained the **slack value** is **greater than 0 and less than 1.**

Hold report-

```
icc2_shell> report_timing -significant_digits 6 -delay_type min
****************************************
Report : timing
        -path_type full
        -delay_type min
        -max_paths 1
        -report_by design
Design : mux_8x1_8bit
Version: V-2023.12-SP4
Date   : Mon Jul  7 15:41:17 2025
****************************************
```

Startpoint: sel_reg_reg[2] (rising edge-triggered flip-flop clocked by clk)

Endpoint: out_mux_reg[7] (rising edge-triggered flip-flop clocked by clk)

Mode: func

Corner: nom

Scenario: func::nom

Path Group: clk

Path Type: **min**

Point	Incr	Path
clock clk (rise edge)	0.000000	0.000000
clock network delay (propagated)	0.001087	0.001087
sel_reg_reg[2]/CLK (DFFARX1_RVT)	0.000000	0.001087 r
sel_reg_reg[2]/QN (DFFARX1_RVT)	0.077515	0.078602 r
U316/Y (OR2X1_RVT)	0.035458	0.114059 r
U323/Y (AND2X1_RVT)	0.025997	0.140057 r
out_mux_reg[7]/D (DFFASX1_RVT)	0.000019	0.140076 r
data arrival time		0.140076
clock clk (rise edge)	0.000000	0.000000
clock network delay (propagated)	0.071621	0.071621
out_mux_reg[7]/CLK (DFFASX1_RVT)	0.000000	0.071621 r
library hold time	0.035023	0.106645
data required time		0.106645
data required time		0.106645
data arrival time		-0.140076

slack (MET) **0.033431**

1

Report_qor

icc2_shell> report_qor

****

Report : qor

Design : mux_8x1_8bit

Version: V-2023.12-SP4

Date : Mon Jul 7 15:44:31 2025

****

Scenario 'func::nom'

Timing Path Group '**in2reg_default**'

Levels of Logic: 1

Critical Path Length: 1.09

Critical Path Slack: -0.18

Critical Path Clk Period: 1.20

Total Negative Slack: -12.58

No. of Violating Paths: 116

Scenario 'func::nom'

Timing Path Group '**reg2out_default**'

Levels of Logic: 0

Critical Path Length: 0.07
Critical Path Slack: 0.01
Critical Path Clk Period: 1.20
Total Negative Slack: 0.00
No. of Violating Paths: 0

Scenario 'func::nom'
Timing Path Group 'clk'

Levels of Logic: 8
Critical Path Length: 0.40
Critical Path Slack: 0.56
Critical Path Clk Period: 1.20
Total Negative Slack: 0.00
No. of Violating Paths: 0
Worst Hold Violation: 0.00
Total Hold Violation: 0.00
No. of Hold Violations: 0

Cell Count

Hierarchical Cell Count: 0
Hierarchical Port Count: 0
Leaf Cell Count: 294

Buf/Inv Cell Count: 111

Buf Cell Count: 73

Inv Cell Count: 38

Combinational Cell Count: 219

Single-bit Isolation Cell Count: 0

Multi-bit Isolation Cell Count: 0

Isolation Cell Banking Ratio: 0.00%

Single-bit Level Shifter Cell Count: 0

Multi-bit Level Shifter Cell Count: 0

Level Shifter Cell Banking Ratio: 0.00%

Single-bit ELS Cell Count: 0

Multi-bit ELS Cell Count: 0

ELS Cell Banking Ratio: 0.00%

Sequential Cell Count: 75

Integrated Clock-Gating Cell Count: 0

Sequential Macro Cell Count: 0

Single-bit Sequential Cell Count: 75

Multi-bit Sequential Cell Count: 0

Sequential Cell Banking Ratio: 0.00%

BitsPerflop: 1.00

Macro Count: 0

Area

Combinational Area: 597.75

Noncombinational Area: 533.70

Buf/Inv Area: 353.77

Total Buffer Area: 281.85

Total Inverter Area: 71.92

Macro/Black Box Area: 0.00

Net Area: 0

Net XLength: 1693.56

Net YLength: 1832.66

Cell Area (netlist): 1131.45

Cell Area (netlist and physical only): 1131.45

Net Length: 3526.22

Design Rules

Total Number of Nets: 368

Nets with Violations: 0

Max Trans Violations: 0

Max Cap Violations: 0

1

Report_design

icc2_shell> report_design

****

Report : design

Design : mux_8x1_8bit

Version: V-2023.12-SP4

Date : Mon Jul 7 15:48:48 2025

****

Total number of std cells in library : 286

Total number of dont_use lib cells : 85

Total number of dont_touch lib cells : 7

Total number of buffers : 11

Total number of inverters : 15

Total number of flip-flops : 106

Total number of latches : 12

Total number of ICGs : 12

Cell Instance Type	Count	Area
TOTAL LEAF CELLS	294	1131.449
unit	294	1131.449
Standard cells	294	1131.449
unit	294	1131.449
Hard macro cells	0	0.000
unit	294	1131.449
Soft macro cells	0	0.000
unit	294	1131.449
Always on cells	0	0.000
unit	294	1131.449
Physical only	0	0.000
unit	294	1131.449

Fixed cells	0	0.000
unit	294	1131.449
Moveable cells	294	1131.449
unit	294	1131.449
Placed cells	294	1131.449
unit	294	1131.449
Sequential	75	533.702
unit	294	1131.449
Buffer/inverter	111	353.768
unit	294	1131.449
ICG cells	0	0.000
unit	294	1131.449

Logic Hierarchies : 0
Design Masters count : 11
Total Flat nets count : 368
Total FloatingNets count : 0
Total no of Ports : 79
Number of Master Clocks in design : 1
Number of Generated Clocks in design : 0
Number of Path Groups in design : 7 (1 of them Non Default)
Number of Scan Chains in design : 0
List of Modes : func
List of Corners : nom
List of Scenarios : func::nom

Core Area : 1353.063

Chip Area : 2005.607

Total Site Row Area : 1353.063

Number of Blockages : 0

Total area of Blockages : 0.000

Number of Power Domains : 1

Number of Voltage Areas : 1

Number of Group Bounds : 0

Number of Exclusive MoveBounds : 0

Number of Hard or Soft MoveBounds : 0

Number of Multibit Registers : 0

Number of Multibit LS/ISO Cells : 0

Number of Top Level RP Groups : 0

Number of Tech Layers : 71 (61 of them have unknown routing dir.)

Total wire length : 3526.22 micron

Total number of wires : 2123

Total number of contacts : 2867

1

Compiler_ultra

dc_shell> compile_ultra

RM-Info: Completed script /ve/po_home/b23n_sarif_727/internship_may_jun_25/simulation1/rm_setup/dc_setup.tcl

Loading db file '/ve/po_home/b23n_sarif_727/internship_may_jun_25/ref/lib/stdcell_rvt/saed32rvt_ff1p16v125c.db'

Loading db file '/ve/po_home/b23n_sarif_727/internship_may_jun_25/ref/lib/stdcell_rvt/saed32rvt_tt0p78vn40c.db'

Loading db file '/ve/po_home/b23n_sarif_727/internship_may_jun_25/ref/lib/stdcell_rvt/saed32rvt_ss0p7vn40c.db'

Loading db file '/usr/synopsys/syn/V-2023.12-SP4/libraries/syn/dw_foundation.sldb'

Running PRESTO HDLC

Compiling source file ./mux_8x1_8bit.sv

Presto compilation completed successfully.

Loading db file '/usr/synopsys/syn/V-2023.12-SP4/libraries/syn/gtech.db'

Loading db file '/usr/synopsys/syn/V-2023.12-SP4/libraries/syn/standard.sldb'

  Loading link library 'gtech'

Running PRESTO HDLC

Warning: ./mux_8x1_8bit.sv:50: DEFAULT branch of CASE statement cannot be reached. (ELAB-311)

Statistics for case statements in always block at line 40 in file

        './mux_8x1_8bit.sv'

```
===============================================
|          Line          |  full/ parallel  |
===============================================
|           41           |    user/user     |
===============================================
```

Inferred memory devices in process

        in routine mux_8x1_8bit line 16 in file

                './mux_8x1_8bit.sv'.

```
=============================================================================
===============
|    Register Name    |   Type    | Width | Bus | MB | AR | AS | SR | SS | ST |
```

```
================================================================
===============
|    sel_reg_reg     | Flip-flop |   3   |  Y  | N | Y  | N | N  | N  | N  |
|    in0_reg_reg     | Flip-flop |   8   |  Y  | N | Y  | N | N  | N  | N  |
|    in1_reg_reg     | Flip-flop |   8   |  Y  | N | Y  | N | N  | N  | N  |
|    in2_reg_reg     | Flip-flop |   8   |  Y  | N | Y  | N | N  | N  | N  |
|    in3_reg_reg     | Flip-flop |   8   |  Y  | N | Y  | N | N  | N  | N  |
|    in4_reg_reg     | Flip-flop |   8   |  Y  | N | Y  | N | N  | N  | N  |
|    in5_reg_reg     | Flip-flop |   8   |  Y  | N | Y  | N | N  | N  | N  |
|    in6_reg_reg     | Flip-flop |   8   |  Y  | N | Y  | N | N  | N  | N  |
|    in7_reg_reg     | Flip-flop |   8   |  Y  | N | Y  | N | N  | N  | N  |
================================================================
===============

Inferred memory devices in process
        in routine mux_8x1_8bit line 57 in file
                './mux_8x1_8bit.sv'.
================================================================
===============
|   Register Name    |   Type    | Width | Bus | MB | AR | AS | SR | SS | ST |
================================================================
===============
|    out_mux_reg     | Flip-flop |   8   |  Y  | N | Y  | N | N  | N  | N  |
================================================================
===============
Presto compilation completed successfully. (mux_8x1_8bit)
Elaborated 1 design.
Current design is now 'mux_8x1_8bit'.
Current design is 'mux_8x1_8bit'.
```

```
============================================================
====================================
| Flow Information                                         |
------------------------------------------------------------------------------------------
| Flow         | Design Compiler WLM                                 |
| Command Line | compile_ultra                                       |
============================================================
====================================
| Design Information                  | Value                    |
============================================================
====================================
| Number of Scenarios                 | 0                        |
| Leaf Cell Count                     | 109                      |
| Number of User Hierarchies          | 0                        |
| Sequential Cell Count               | 75                       |
| Macro Count                         | 0                        |
| Number of Power Domains             | 0                        |
| Number of Path Groups               | 2                        |
| Number of VT Class                  | 1                        |
| Number of Clocks                    | 1                        |
| Number of Dont Touch Cells          | 16                       |
| Number of Dont Touch Nets           | 0                        |
| Number of Size Only Cells           | 0                        |
| Design with UPF Data                | false                    |
============================================================
====================================

Information: Sequential output inversion is enabled.  SVF file must be used for formal verification. (OPT-1208)

  Simplifying Design 'mux_8x1_8bit'
```

Mux_8x1_8bit.sdc file

```
create_clock -period 2.5 [get_ports clk]
set_clock_uncertainty -setup 0.05 [get_clocks clk]
set_clock_latency -source -max 0.0700 [get_clocks clk]
set_clock_latency -max 0.400 [get_clocks clk]
set_clock_transition -max 0.25 [get_clocks clk]

set_input_delay -max 1 -clock clk [all_inputs]
set_input_transition -max 1 [all_inputs]

set_output_delay -max 1 -clock clk [all_outputs]
set_load -max 1 [all_outputs]
```

Mapping netlist file

```
////////////////////////////////////////////////////////////
// Created by: Synopsys DC Ultra(TM) in wire load mode
// Version   : V-2023.12-SP4
// Date      : Mon Jul  7 16:02:52 2025
////////////////////////////////////////////////////////////



module mux_8x1_8bit ( clk, rst, in0, in1, in2, in3, in4, in5, in6, in7, sel,
        out_mux );
  input [7:0] in0;
  input [7:0] in1;
  input [7:0] in2;
  input [7:0] in3;
  input [7:0] in4;
```

```
input [7:0] in5;
input [7:0] in6;
input [7:0] in7;
input [2:0] sel;
output [7:0] out_mux;
input clk, rst;
wire   n318, n319, n320, n321, n322, n323, n324, n325, n326, n327, n328,
       n329, n330, n331, n332, n333, n334, n335, n336, n337, n338, n339,
       n340, n341, n342, n343, n344, n345, n346, n347, n348, n349, n350,
       n351, n352, n353, n354, n355, n356, n357, n358, n359, n360, n361,
       n362, n363, n364, n365, n366, n367, n368, n369, n370, n371, n372,
       n373, n374, n105, n106, n107, n108, n109, n110, n111, n112, n113,
       n114, n115, n116, n117, n118, n119, n120, n121, n122, n123, n124,
       n125, n126, n127, n128, n129, n130, n131, n132, n133, n134, n135,
       n136, n137, n138, n139, n140, n141, n142, n143, n144, n145, n146,
       n147, n148, n149, n150, n151, n152, n153, n154, n155, n156, n157,
       n158, n159, n160, n161, n162, n163, n164, n165, n166, n167, n168,
       n169, n170, n171, n172, n173, n174, n175, n176, n177, n178, n179,
       n180, n181, n182, n183, n184, n185, n186, n187, n188, n189, n190,
       n191, n192, n193, n194, n195, n196, n197, n198, n199, n200, n201,
       n202, n203, n204, n205, n206, n207, n208, n209, n210, n211, n212,
       n213, n214, n215, n216, n217, n218, n219, n220, n221, n222, n223,
       n224, n225, n226, n227, n228, n229, n230, n231, n232, n233, n234,
       n235, n236, n237, n238, n239, n240, n298, n299, n300, n301, n302,
       n303, n304, n305, n306, n307, n308, n309, n310, n311, n312, n313,
       n314, n315, n316, n317;
wire   [7:0] in0_reg;
```

```
wire   [7:0] in1_reg;
wire   [7:0] in2_reg;
wire   [7:0] in3_reg;
wire   [7:0] in4_reg;
wire   [7:0] in5_reg;
wire   [7:0] in6_reg;
wire   [7:0] in7_reg;
wire   [2:0] sel_reg;

DFFARX1_RVT \sel_reg_reg[2]  ( .D(sel[2]), .CLK(clk), .RSTB(n306), .Q(n300),
     .QN(n301) );
DFFARX1_RVT \sel_reg_reg[1]  ( .D(sel[1]), .CLK(clk), .RSTB(n306), .Q(
     sel_reg[1]), .QN(n299) );
DFFARX1_RVT \sel_reg_reg[0]  ( .D(sel[0]), .CLK(clk), .RSTB(n306), .Q(
     sel_reg[0]), .QN(n298) );
DFFARX1_RVT \in0_reg_reg[7]  ( .D(n318), .CLK(clk), .RSTB(n305), .Q(
     in0_reg[7]) );
DFFARX1_RVT \in0_reg_reg[6]  ( .D(n319), .CLK(clk), .RSTB(n305), .Q(
     in0_reg[6]) );
DFFARX1_RVT \in0_reg_reg[5]  ( .D(n320), .CLK(clk), .RSTB(n305), .Q(
     in0_reg[5]) );
DFFARX1_RVT \in0_reg_reg[4]  ( .D(n321), .CLK(clk), .RSTB(n305), .Q(
     in0_reg[4]) );
DFFARX1_RVT \in0_reg_reg[3]  ( .D(n322), .CLK(clk), .RSTB(n305), .Q(
     in0_reg[3]) );
DFFARX1_RVT \in0_reg_reg[2]  ( .D(n323), .CLK(clk), .RSTB(n305), .Q(
     in0_reg[2]) );
```

```
DFFARX1_RVT \in0_reg_reg[1]  ( .D(n324), .CLK(clk), .RSTB(n305), .Q(
    in0_reg[1]) );
DFFARX1_RVT \in0_reg_reg[0]  ( .D(n325), .CLK(clk), .RSTB(n305), .Q(
    in0_reg[0]) );
DFFARX1_RVT \in1_reg_reg[7]  ( .D(n326), .CLK(clk), .RSTB(n305), .Q(
    in1_reg[7]) );
DFFARX1_RVT \in1_reg_reg[6]  ( .D(n327), .CLK(clk), .RSTB(n305), .Q(
    in1_reg[6]) );
DFFARX1_RVT \in1_reg_reg[5]  ( .D(n328), .CLK(clk), .RSTB(n305), .Q(
    in1_reg[5]) );
DFFARX1_RVT \in1_reg_reg[4]  ( .D(n329), .CLK(clk), .RSTB(n305), .Q(
    in1_reg[4]) );
DFFARX1_RVT \in1_reg_reg[3]  ( .D(n330), .CLK(clk), .RSTB(n304), .Q(
    in1_reg[3]) );
DFFARX1_RVT \in1_reg_reg[2]  ( .D(n331), .CLK(clk), .RSTB(n304), .Q(
    in1_reg[2]) );
DFFARX1_RVT \in1_reg_reg[1]  ( .D(n332), .CLK(clk), .RSTB(n304), .Q(
    in1_reg[1]) );
DFFARX1_RVT \in1_reg_reg[0]  ( .D(n333), .CLK(clk), .RSTB(n304), .Q(
    in1_reg[0]) );
DFFARX1_RVT \in2_reg_reg[7]  ( .D(n334), .CLK(clk), .RSTB(n304), .Q(
    in2_reg[7]) );
DFFARX1_RVT \in2_reg_reg[6]  ( .D(n335), .CLK(clk), .RSTB(n304), .Q(
    in2_reg[6]) );
DFFARX1_RVT \in2_reg_reg[5]  ( .D(n336), .CLK(clk), .RSTB(n304), .Q(
    in2_reg[5]) );
DFFARX1_RVT \in2_reg_reg[4]  ( .D(n337), .CLK(clk), .RSTB(n304), .Q(
```

```
    in2_reg[4]) );
DFFARX1_RVT \in2_reg_reg[3]  ( .D(n338), .CLK(clk), .RSTB(n304), .Q(
    in2_reg[3]) );
DFFARX1_RVT \in2_reg_reg[2]  ( .D(n339), .CLK(clk), .RSTB(n304), .Q(
    in2_reg[2]) );
DFFARX1_RVT \in2_reg_reg[1]  ( .D(n340), .CLK(clk), .RSTB(n304), .Q(
    in2_reg[1]) );
DFFARX1_RVT \in2_reg_reg[0]  ( .D(n341), .CLK(clk), .RSTB(n304), .Q(
    in2_reg[0]) );
DFFARX1_RVT \in3_reg_reg[7]  ( .D(n342), .CLK(clk), .RSTB(n303), .Q(
    in3_reg[7]) );
DFFARX1_RVT \in3_reg_reg[6]  ( .D(n343), .CLK(clk), .RSTB(n303), .Q(
    in3_reg[6]) );
DFFARX1_RVT \in3_reg_reg[5]  ( .D(n344), .CLK(clk), .RSTB(n303), .Q(
    in3_reg[5]) );
DFFARX1_RVT \in3_reg_reg[4]  ( .D(n345), .CLK(clk), .RSTB(n303), .Q(
    in3_reg[4]) );
DFFARX1_RVT \in3_reg_reg[3]  ( .D(n346), .CLK(clk), .RSTB(n303), .Q(
    in3_reg[3]) );
DFFARX1_RVT \in3_reg_reg[2]  ( .D(n347), .CLK(clk), .RSTB(n303), .Q(
    in3_reg[2]) );
DFFARX1_RVT \in3_reg_reg[1]  ( .D(n348), .CLK(clk), .RSTB(n303), .Q(
    in3_reg[1]) );
DFFARX1_RVT \in3_reg_reg[0]  ( .D(n349), .CLK(clk), .RSTB(n303), .Q(
    in3_reg[0]) );
DFFARX1_RVT \in4_reg_reg[7]  ( .D(n350), .CLK(clk), .RSTB(n303), .Q(
    in4_reg[7]) );
```

```
DFFARX1_RVT \in4_reg_reg[6]  ( .D(n351), .CLK(clk), .RSTB(n303), .Q(
    in4_reg[6]) );
DFFARX1_RVT \in4_reg_reg[5]  ( .D(n352), .CLK(clk), .RSTB(n303), .Q(
    in4_reg[5]) );
DFFARX1_RVT \in4_reg_reg[4]  ( .D(n353), .CLK(clk), .RSTB(n303), .Q(
    in4_reg[4]) );
DFFARX1_RVT \in4_reg_reg[3]  ( .D(n354), .CLK(clk), .RSTB(n306), .Q(
    in4_reg[3]) );
DFFARX1_RVT \in4_reg_reg[2]  ( .D(n355), .CLK(clk), .RSTB(n306), .Q(
    in4_reg[2]) );
DFFARX1_RVT \in4_reg_reg[1]  ( .D(n356), .CLK(clk), .RSTB(n306), .Q(
    in4_reg[1]) );
DFFARX1_RVT \in4_reg_reg[0]  ( .D(n357), .CLK(clk), .RSTB(n306), .Q(
    in4_reg[0]) );
DFFARX1_RVT \in5_reg_reg[7]  ( .D(n358), .CLK(clk), .RSTB(n306), .Q(
    in5_reg[7]) );
DFFARX1_RVT \in5_reg_reg[6]  ( .D(n359), .CLK(clk), .RSTB(n306), .Q(
    in5_reg[6]) );
DFFARX1_RVT \in5_reg_reg[5]  ( .D(n360), .CLK(clk), .RSTB(n306), .Q(
    in5_reg[5]) );
DFFARX1_RVT \in5_reg_reg[4]  ( .D(n361), .CLK(clk), .RSTB(n306), .Q(
    in5_reg[4]) );
DFFARX1_RVT \in5_reg_reg[3]  ( .D(n362), .CLK(clk), .RSTB(n306), .Q(
    in5_reg[3]) );
DFFARX1_RVT \in5_reg_reg[2]  ( .D(n363), .CLK(clk), .RSTB(n306), .Q(
    in5_reg[2]) );
DFFARX1_RVT \in5_reg_reg[1]  ( .D(n364), .CLK(clk), .RSTB(n305), .Q(
```

```
    in5_reg[1]) );
DFFARX1_RVT \in5_reg_reg[0]  ( .D(n365), .CLK(clk), .RSTB(n303), .Q(
    in5_reg[0]) );
DFFARX1_RVT \in6_reg_reg[7]  ( .D(n366), .CLK(clk), .RSTB(n302), .Q(
    in6_reg[7]) );
DFFARX1_RVT \in6_reg_reg[6]  ( .D(n367), .CLK(clk), .RSTB(n302), .Q(
    in6_reg[6]) );
DFFARX1_RVT \in6_reg_reg[5]  ( .D(n368), .CLK(clk), .RSTB(n302), .Q(
    in6_reg[5]) );
DFFARX1_RVT \in6_reg_reg[4]  ( .D(n369), .CLK(clk), .RSTB(n302), .Q(
    in6_reg[4]) );
DFFARX1_RVT \in6_reg_reg[3]  ( .D(n370), .CLK(clk), .RSTB(n302), .Q(
    in6_reg[3]) );
DFFARX1_RVT \in6_reg_reg[2]  ( .D(n371), .CLK(clk), .RSTB(n302), .Q(
    in6_reg[2]) );
DFFARX1_RVT \in6_reg_reg[1]  ( .D(n372), .CLK(clk), .RSTB(n302), .Q(
    in6_reg[1]) );
DFFARX1_RVT \in6_reg_reg[0]  ( .D(n373), .CLK(clk), .RSTB(n302), .Q(
    in6_reg[0]) );
DFFARX1_RVT \in7_reg_reg[7]  ( .D(n374), .CLK(clk), .RSTB(n302), .Q(
    in7_reg[7]) );
DFFARX1_RVT \in7_reg_reg[5]  ( .D(in7[5]), .CLK(clk), .RSTB(n302), .Q(
    in7_reg[5]) );
DFFARX1_RVT \in7_reg_reg[4]  ( .D(in7[4]), .CLK(clk), .RSTB(n302), .Q(
    in7_reg[4]) );
DFFARX1_RVT \in7_reg_reg[3]  ( .D(in7[3]), .CLK(clk), .RSTB(n317), .Q(
    in7_reg[3]) );
```

```
DFFARX1_RVT \in7_reg_reg[2]  ( .D(in7[2]), .CLK(clk), .RSTB(n317), .Q(
      in7_reg[2]) );
DFFARX1_RVT \in7_reg_reg[1]  ( .D(in7[1]), .CLK(clk), .RSTB(n308), .Q(
      in7_reg[1]) );
DFFARX1_RVT \in7_reg_reg[0]  ( .D(in7[0]), .CLK(clk), .RSTB(n308), .Q(
      in7_reg[0]) );
DFFASX1_RVT \out_mux_reg[5]  ( .D(n314), .CLK(clk), .SETB(n317), .QN(
      out_mux[5]) );
DFFASX1_RVT \out_mux_reg[2]  ( .D(n311), .CLK(clk), .SETB(n307), .QN(
      out_mux[2]) );
DFFASX1_RVT \out_mux_reg[6]  ( .D(n315), .CLK(clk), .SETB(n308), .QN(
      out_mux[6]) );
DFFASX1_RVT \out_mux_reg[3]  ( .D(n312), .CLK(clk), .SETB(n308), .QN(
      out_mux[3]) );
DFFASX1_RVT \out_mux_reg[0]  ( .D(n309), .CLK(clk), .SETB(n307), .QN(
      out_mux[0]) );
DFFASX1_RVT \out_mux_reg[4]  ( .D(n313), .CLK(clk), .SETB(n317), .QN(
      out_mux[4]) );
DFFASX1_RVT \out_mux_reg[1]  ( .D(n310), .CLK(clk), .SETB(n308), .QN(
      out_mux[1]) );
DFFASX1_RVT \out_mux_reg[7]  ( .D(n316), .CLK(clk), .SETB(n307), .QN(
      out_mux[7]) );
DFFASX1_RVT \in7_reg_reg[6]  ( .D(n240), .CLK(clk), .SETB(n302), .QN(
      in7_reg[6]) );
OR2X1_RVT U115 ( .A1(n298), .A2(n299), .Y(n124) );
INVX1_RVT U116 ( .A(n173), .Y(n113) );
INVX1_RVT U117 ( .A(n124), .Y(n230) );
```

```
INVX1_RVT U118 ( .A(sel_reg[1]), .Y(n106) );
INVX0_RVT U119 ( .A(n204), .Y(n205) );
INVX0_RVT U120 ( .A(n171), .Y(n172) );
INVX0_RVT U121 ( .A(n129), .Y(n130) );
INVX0_RVT U122 ( .A(n149), .Y(n150) );
INVX0_RVT U123 ( .A(n225), .Y(n226) );
INVX0_RVT U124 ( .A(n123), .Y(n197) );
INVX0_RVT U125 ( .A(n123), .Y(n228) );
INVX0_RVT U126 ( .A(n122), .Y(n217) );
INVX0_RVT U127 ( .A(n122), .Y(n227) );
INVX0_RVT U128 ( .A(n113), .Y(n219) );
INVX0_RVT U129 ( .A(n123), .Y(n218) );
INVX0_RVT U130 ( .A(n113), .Y(n198) );
INVX0_RVT U131 ( .A(n122), .Y(n196) );
INVX0_RVT U132 ( .A(n113), .Y(n229) );
NBUFFX2_RVT U133 ( .A(n306), .Y(n304) );
INVX0_RVT U134 ( .A(n207), .Y(n123) );
INVX0_RVT U135 ( .A(n206), .Y(n122) );
NBUFFX2_RVT U136 ( .A(n303), .Y(n305) );
NBUFFX2_RVT U137 ( .A(n302), .Y(n306) );
INVX0_RVT U138 ( .A(n124), .Y(n220) );
INVX0_RVT U139 ( .A(n124), .Y(n199) );
INVX0_RVT U140 ( .A(n300), .Y(n214) );
INVX0_RVT U141 ( .A(n301), .Y(n237) );
INVX0_RVT U142 ( .A(sel_reg[0]), .Y(n105) );
INVX0_RVT U143 ( .A(in7[6]), .Y(n240) );
OR4X1_RVT U144 ( .A1(n234), .A2(n233), .A3(n232), .A4(n231), .Y(n235) );
```

OR4X1_RVT U145 (.A1(n134), .A2(n133), .A3(n132), .A4(n131), .Y(n135));

INVX0_RVT U146 (.A(rst), .Y(n317));

NBUFFX2_RVT U147 (.A(in7[7]), .Y(n374));

NBUFFX2_RVT U148 (.A(in6[0]), .Y(n373));

NBUFFX2_RVT U149 (.A(in6[1]), .Y(n372));

NBUFFX2_RVT U150 (.A(in6[2]), .Y(n371));

NBUFFX2_RVT U151 (.A(in6[3]), .Y(n370));

NBUFFX2_RVT U152 (.A(in6[4]), .Y(n369));

NBUFFX2_RVT U153 (.A(in6[5]), .Y(n368));

NBUFFX2_RVT U154 (.A(in6[6]), .Y(n367));

NBUFFX2_RVT U155 (.A(in6[7]), .Y(n366));

NBUFFX2_RVT U156 (.A(in5[0]), .Y(n365));

NBUFFX2_RVT U157 (.A(in5[1]), .Y(n364));

NBUFFX2_RVT U158 (.A(in5[2]), .Y(n363));

NBUFFX2_RVT U159 (.A(in5[3]), .Y(n362));

NBUFFX2_RVT U160 (.A(in5[4]), .Y(n361));

NBUFFX2_RVT U161 (.A(in5[5]), .Y(n360));

NBUFFX2_RVT U162 (.A(in5[6]), .Y(n359));

NBUFFX2_RVT U163 (.A(in5[7]), .Y(n358));

NBUFFX2_RVT U164 (.A(in4[0]), .Y(n357));

NBUFFX2_RVT U165 (.A(in4[1]), .Y(n356));

NBUFFX2_RVT U166 (.A(in4[2]), .Y(n355));

NBUFFX2_RVT U167 (.A(in4[3]), .Y(n354));

NBUFFX2_RVT U168 (.A(in4[4]), .Y(n353));

NBUFFX2_RVT U169 (.A(in4[5]), .Y(n352));

NBUFFX2_RVT U170 (.A(in4[6]), .Y(n351));

NBUFFX2_RVT U171 (.A(in4[7]), .Y(n350));

```
NBUFFX2_RVT U172 ( .A(in3[0]), .Y(n349) );
NBUFFX2_RVT U173 ( .A(in3[1]), .Y(n348) );
NBUFFX2_RVT U174 ( .A(in3[2]), .Y(n347) );
NBUFFX2_RVT U175 ( .A(in3[3]), .Y(n346) );
NBUFFX2_RVT U176 ( .A(in3[4]), .Y(n345) );
NBUFFX2_RVT U177 ( .A(in3[5]), .Y(n344) );
NBUFFX2_RVT U178 ( .A(in3[6]), .Y(n343) );
NBUFFX2_RVT U179 ( .A(in3[7]), .Y(n342) );
NBUFFX2_RVT U180 ( .A(in2[0]), .Y(n341) );
NBUFFX2_RVT U181 ( .A(in2[1]), .Y(n340) );
NBUFFX2_RVT U182 ( .A(in2[2]), .Y(n339) );
NBUFFX2_RVT U183 ( .A(in2[3]), .Y(n338) );
NBUFFX2_RVT U184 ( .A(in2[4]), .Y(n337) );
NBUFFX2_RVT U185 ( .A(in2[5]), .Y(n336) );
NBUFFX2_RVT U186 ( .A(in2[6]), .Y(n335) );
NBUFFX2_RVT U187 ( .A(in2[7]), .Y(n334) );
NBUFFX2_RVT U188 ( .A(in1[0]), .Y(n333) );
NBUFFX2_RVT U189 ( .A(in1[1]), .Y(n332) );
NBUFFX2_RVT U190 ( .A(in1[2]), .Y(n331) );
NBUFFX2_RVT U191 ( .A(in1[3]), .Y(n330) );
NBUFFX2_RVT U192 ( .A(in1[4]), .Y(n329) );
NBUFFX2_RVT U193 ( .A(in1[5]), .Y(n328) );
NBUFFX2_RVT U194 ( .A(in1[6]), .Y(n327) );
NBUFFX2_RVT U195 ( .A(in1[7]), .Y(n326) );
NBUFFX2_RVT U196 ( .A(in0[0]), .Y(n325) );
NBUFFX2_RVT U197 ( .A(in0[1]), .Y(n324) );
NBUFFX2_RVT U198 ( .A(in0[2]), .Y(n323) );
```

```
NBUFFX2_RVT U199 ( .A(in0[3]), .Y(n322) );
NBUFFX2_RVT U200 ( .A(in0[4]), .Y(n321) );
NBUFFX2_RVT U201 ( .A(in0[5]), .Y(n320) );
NBUFFX2_RVT U202 ( .A(in0[6]), .Y(n319) );
NBUFFX2_RVT U203 ( .A(in0[7]), .Y(n318) );
NBUFFX2_RVT U204 ( .A(n317), .Y(n307) );
NBUFFX2_RVT U205 ( .A(n307), .Y(n302) );
NBUFFX2_RVT U206 ( .A(n307), .Y(n303) );
NBUFFX2_RVT U207 ( .A(n317), .Y(n308) );
AND2X1_RVT U208 ( .A1(n105), .A2(n106), .Y(n206) );
AND2X1_RVT U209 ( .A1(n196), .A2(in0_reg[0]), .Y(n110) );
AND2X1_RVT U210 ( .A1(sel_reg[1]), .A2(n105), .Y(n207) );
AND2X1_RVT U211 ( .A1(n197), .A2(in2_reg[0]), .Y(n109) );
AND2X1_RVT U212 ( .A1(sel_reg[0]), .A2(n106), .Y(n173) );
AND2X1_RVT U213 ( .A1(n198), .A2(in1_reg[0]), .Y(n108) );
AND2X1_RVT U214 ( .A1(n199), .A2(in3_reg[0]), .Y(n107) );
OR4X1_RVT U215 ( .A1(n110), .A2(n109), .A3(n108), .A4(n107), .Y(n111) );
INVX0_RVT U216 ( .A(n111), .Y(n112) );
OR2X1_RVT U217 ( .A1(n237), .A2(n112), .Y(n121) );
AND2X1_RVT U218 ( .A1(n227), .A2(in4_reg[0]), .Y(n117) );
AND2X1_RVT U219 ( .A1(n228), .A2(in6_reg[0]), .Y(n116) );
AND2X1_RVT U220 ( .A1(n229), .A2(in5_reg[0]), .Y(n115) );
AND2X1_RVT U221 ( .A1(n230), .A2(in7_reg[0]), .Y(n114) );
OR4X1_RVT U222 ( .A1(n117), .A2(n116), .A3(n115), .A4(n114), .Y(n118) );
INVX0_RVT U223 ( .A(n118), .Y(n119) );
OR2X1_RVT U224 ( .A1(n214), .A2(n119), .Y(n120) );
AND2X1_RVT U225 ( .A1(n121), .A2(n120), .Y(n309) );
```

```
AND2X1_RVT U226 ( .A1(n217), .A2(in4_reg[1]), .Y(n128) );
AND2X1_RVT U227 ( .A1(n218), .A2(in6_reg[1]), .Y(n127) );
AND2X1_RVT U228 ( .A1(n219), .A2(in5_reg[1]), .Y(n126) );
AND2X1_RVT U229 ( .A1(n220), .A2(in7_reg[1]), .Y(n125) );
OR4X1_RVT U230 ( .A1(n128), .A2(n127), .A3(n126), .A4(n125), .Y(n129) );
OR2X1_RVT U231 ( .A1(n301), .A2(n130), .Y(n138) );
AND2X1_RVT U232 ( .A1(n227), .A2(in0_reg[1]), .Y(n134) );
AND2X1_RVT U233 ( .A1(n228), .A2(in2_reg[1]), .Y(n133) );
AND2X1_RVT U234 ( .A1(n229), .A2(in1_reg[1]), .Y(n132) );
AND2X1_RVT U235 ( .A1(n230), .A2(in3_reg[1]), .Y(n131) );
INVX0_RVT U236 ( .A(n135), .Y(n136) );
OR2X1_RVT U237 ( .A1(n300), .A2(n136), .Y(n137) );
AND2X1_RVT U238 ( .A1(n138), .A2(n137), .Y(n310) );
AND2X1_RVT U239 ( .A1(n196), .A2(in4_reg[2]), .Y(n142) );
AND2X1_RVT U240 ( .A1(n197), .A2(in6_reg[2]), .Y(n141) );
AND2X1_RVT U241 ( .A1(n198), .A2(in5_reg[2]), .Y(n140) );
AND2X1_RVT U242 ( .A1(n199), .A2(in7_reg[2]), .Y(n139) );
OR4X1_RVT U243 ( .A1(n142), .A2(n141), .A3(n140), .A4(n139), .Y(n143) );
INVX0_RVT U244 ( .A(n143), .Y(n144) );
OR2X1_RVT U245 ( .A1(n214), .A2(n144), .Y(n152) );
AND2X1_RVT U246 ( .A1(n217), .A2(in0_reg[2]), .Y(n148) );
AND2X1_RVT U247 ( .A1(n218), .A2(in2_reg[2]), .Y(n147) );
AND2X1_RVT U248 ( .A1(n219), .A2(in1_reg[2]), .Y(n146) );
AND2X1_RVT U249 ( .A1(n220), .A2(in3_reg[2]), .Y(n145) );
OR4X1_RVT U250 ( .A1(n148), .A2(n147), .A3(n146), .A4(n145), .Y(n149) );
OR2X1_RVT U251 ( .A1(n300), .A2(n150), .Y(n151) );
AND2X1_RVT U252 ( .A1(n152), .A2(n151), .Y(n311) );
```

```
AND2X1_RVT U253 ( .A1(n196), .A2(in0_reg[3]), .Y(n156) );
AND2X1_RVT U254 ( .A1(n197), .A2(in2_reg[3]), .Y(n155) );
AND2X1_RVT U255 ( .A1(n198), .A2(in1_reg[3]), .Y(n154) );
AND2X1_RVT U256 ( .A1(n199), .A2(in3_reg[3]), .Y(n153) );
OR4X1_RVT U257 ( .A1(n156), .A2(n155), .A3(n154), .A4(n153), .Y(n157) );
INVX0_RVT U258 ( .A(n157), .Y(n158) );
OR2X1_RVT U259 ( .A1(n237), .A2(n158), .Y(n166) );
AND2X1_RVT U260 ( .A1(n227), .A2(in4_reg[3]), .Y(n162) );
AND2X1_RVT U261 ( .A1(n228), .A2(in6_reg[3]), .Y(n161) );
AND2X1_RVT U262 ( .A1(n229), .A2(in5_reg[3]), .Y(n160) );
AND2X1_RVT U263 ( .A1(n230), .A2(in7_reg[3]), .Y(n159) );
OR4X1_RVT U264 ( .A1(n162), .A2(n161), .A3(n160), .A4(n159), .Y(n163) );
INVX0_RVT U265 ( .A(n163), .Y(n164) );
OR2X1_RVT U266 ( .A1(n214), .A2(n164), .Y(n165) );
AND2X1_RVT U267 ( .A1(n166), .A2(n165), .Y(n312) );
AND2X1_RVT U268 ( .A1(n217), .A2(in4_reg[4]), .Y(n170) );
AND2X1_RVT U269 ( .A1(n218), .A2(in6_reg[4]), .Y(n169) );
AND2X1_RVT U270 ( .A1(n219), .A2(in5_reg[4]), .Y(n168) );
AND2X1_RVT U271 ( .A1(n220), .A2(in7_reg[4]), .Y(n167) );
OR4X1_RVT U272 ( .A1(n170), .A2(n169), .A3(n168), .A4(n167), .Y(n171) );
OR2X1_RVT U273 ( .A1(n301), .A2(n172), .Y(n181) );
AND2X1_RVT U274 ( .A1(n227), .A2(in0_reg[4]), .Y(n177) );
AND2X1_RVT U275 ( .A1(n228), .A2(in2_reg[4]), .Y(n176) );
AND2X1_RVT U276 ( .A1(n173), .A2(in1_reg[4]), .Y(n175) );
AND2X1_RVT U277 ( .A1(n230), .A2(in3_reg[4]), .Y(n174) );
OR4X1_RVT U278 ( .A1(n177), .A2(n176), .A3(n175), .A4(n174), .Y(n178) );
INVX0_RVT U279 ( .A(n178), .Y(n179) );
```

```
OR2X1_RVT U280 ( .A1(n300), .A2(n179), .Y(n180) );
AND2X1_RVT U281 ( .A1(n181), .A2(n180), .Y(n313) );
AND2X1_RVT U282 ( .A1(n196), .A2(in4_reg[5]), .Y(n185) );
AND2X1_RVT U283 ( .A1(n197), .A2(in6_reg[5]), .Y(n184) );
AND2X1_RVT U284 ( .A1(n198), .A2(in5_reg[5]), .Y(n183) );
AND2X1_RVT U285 ( .A1(n199), .A2(in7_reg[5]), .Y(n182) );
OR4X1_RVT U286 ( .A1(n185), .A2(n184), .A3(n183), .A4(n182), .Y(n186) );
INVX0_RVT U287 ( .A(n186), .Y(n187) );
OR2X1_RVT U288 ( .A1(n214), .A2(n187), .Y(n195) );
AND2X1_RVT U289 ( .A1(n217), .A2(in0_reg[5]), .Y(n191) );
AND2X1_RVT U290 ( .A1(n218), .A2(in2_reg[5]), .Y(n190) );
AND2X1_RVT U291 ( .A1(n219), .A2(in1_reg[5]), .Y(n189) );
AND2X1_RVT U292 ( .A1(n220), .A2(in3_reg[5]), .Y(n188) );
OR4X1_RVT U293 ( .A1(n191), .A2(n190), .A3(n189), .A4(n188), .Y(n192) );
INVX0_RVT U294 ( .A(n192), .Y(n193) );
OR2X1_RVT U295 ( .A1(n237), .A2(n193), .Y(n194) );
AND2X1_RVT U296 ( .A1(n195), .A2(n194), .Y(n314) );
AND2X1_RVT U297 ( .A1(n196), .A2(in0_reg[6]), .Y(n203) );
AND2X1_RVT U298 ( .A1(n197), .A2(in2_reg[6]), .Y(n202) );
AND2X1_RVT U299 ( .A1(n198), .A2(in1_reg[6]), .Y(n201) );
AND2X1_RVT U300 ( .A1(n199), .A2(in3_reg[6]), .Y(n200) );
OR4X1_RVT U301 ( .A1(n203), .A2(n202), .A3(n201), .A4(n200), .Y(n204) );
OR2X1_RVT U302 ( .A1(n237), .A2(n205), .Y(n216) );
AND2X1_RVT U303 ( .A1(n206), .A2(in4_reg[6]), .Y(n211) );
AND2X1_RVT U304 ( .A1(n207), .A2(in6_reg[6]), .Y(n210) );
AND2X1_RVT U305 ( .A1(n229), .A2(in5_reg[6]), .Y(n209) );
AND2X1_RVT U306 ( .A1(n230), .A2(in7_reg[6]), .Y(n208) );
```

```
  OR4X1_RVT U307 ( .A1(n211), .A2(n210), .A3(n209), .A4(n208), .Y(n212) );
  INVX0_RVT U308 ( .A(n212), .Y(n213) );
  OR2X1_RVT U309 ( .A1(n214), .A2(n213), .Y(n215) );
  AND2X1_RVT U310 ( .A1(n216), .A2(n215), .Y(n315) );
  AND2X1_RVT U311 ( .A1(n217), .A2(in4_reg[7]), .Y(n224) );
  AND2X1_RVT U312 ( .A1(n218), .A2(in6_reg[7]), .Y(n223) );
  AND2X1_RVT U313 ( .A1(n219), .A2(in5_reg[7]), .Y(n222) );
  AND2X1_RVT U314 ( .A1(n220), .A2(in7_reg[7]), .Y(n221) );
  OR4X1_RVT U315 ( .A1(n224), .A2(n223), .A3(n222), .A4(n221), .Y(n225) );
  OR2X1_RVT U316 ( .A1(n301), .A2(n226), .Y(n239) );
  AND2X1_RVT U317 ( .A1(n227), .A2(in0_reg[7]), .Y(n234) );
  AND2X1_RVT U318 ( .A1(n228), .A2(in2_reg[7]), .Y(n233) );
  AND2X1_RVT U319 ( .A1(n229), .A2(in1_reg[7]), .Y(n232) );
  AND2X1_RVT U320 ( .A1(n230), .A2(in3_reg[7]), .Y(n231) );
  INVX0_RVT U321 ( .A(n235), .Y(n236) );
  OR2X1_RVT U322 ( .A1(n237), .A2(n236), .Y(n238) );
  AND2X1_RVT U323 ( .A1(n239), .A2(n238), .Y(n316) );
endmodule
```